Cryptographic Dimension Partitioning

Identity-Bound FFN Confinement and Whole-Model Authorization

Ryan McCormick[1]

[1]Independent Researcher

August 2026

Abstract

Public-key infrastructure made identity administrable: certificates bind keys to names, and signed dereferences compose across organizations that share no other trust. Model artifacts need an analogous infrastructure for *capability*. As trained models are licensed, co-located, and embedded in other parties' systems, what is required is not merely encryption of the artifact but an identity-bound structure *inside* it to which signed grants of scoped capability can unambiguously refer — a public-key infrastructure for model capability. This paper develops the structural basis for that infrastructure. Shared multi-tenant systems lack a structural zero baseline for tenant-scoped learned state and often leave complete-model dispatch to serving configuration. We introduce *Cryptographic Dimension Partitioning* (CDP), which binds an authenticated tenant + sub-id scope to two distinct gate surfaces. At an FFN boundary, a runtime-selected binary mask may act on the pre-MLP input or on the expanded intermediate activation. Both placements give exact zeros at the gated tensor; the intermediate placement additionally aligns the hidden columns of $\mathbf{W}_1$, $\mathbf{b}_1$, and rows of $\mathbf{W}_2$ for the paper's strongest parameter-confinement result. At the whole-model surface, a pre-execution authorization gate treats the complete model as one atomic resource: an authorized request executes the unchanged model, while a rejected request never invokes it.

We verify the gate, gradient, and aligned FFN confinement results in Lean 4 with no `sorry` declarations or project-specific axioms in the headline theorems. A complementary negative theorem shows that placing LayerNorm between the activation and gate breaks $\mathbf{W}_1$ confinement, making placement and path closure explicit preconditions.

The guarantee classes are of different strength, and the empirics are stated in matching strength. Structurally, inactive-state exclusion is exact: in a strict frozen-backbone DistilBERT experiment, every intermediate FFN is gated and only authorized FFN slices plus a private classifier are trainable, and across three seeds the maximum frozen-shared, off-partition parameter, off-partition optimizer-moment, and inactive-classifier delta is exactly zero in every run. Where the network has capacity, partitioning also costs little to nothing measurably: in a five-seed paired sweep at a post-encoder gate, co-trained DistilBERT stays within 0.14–0.38 percentage points of separately trained models across R=8–128 — no paired test survives Bonferroni correction — and in a synthetic capacity-scaling check at R=128 with per-regime width held fixed, all regimes train to 100% owning accuracy, zero of 32,512 wrong-key probes are answered, and support extraction is exact (maximum logit difference 0). Where capacity is genuinely scarce the cost is real and recoverable: strict frozen-backbone owning accuracy decreases from $91.28\% \pm 0.33\%$ at R=1 to $86.21\% \pm 0.08\%$ at R=16, and complete private residual adapters and private experts recover 90.47% and 90.09% mean accuracy, respectively, without modifying the shared backbone or inactive lanes. In an R=8 learned-MoE study on 6,231 held-out examples, separately trained top-1 routers and experts retain exactly equal logits and predictions after packing into one authorized bank, with zero unauthorized dispatch. A token-level extension makes 311,477 held-out routing decisions under execution-selected private continuous prefixes; literal capability-marker text cannot expand authority, and unauthorized dispatch remains zero. The source-visible research preflight checks pre-MLP exclusion and fail-closed whole-model invocation separately. The completed

results support the declared FFN-coordinate, private-state, and atomic model-authorization boundaries; they do not establish multi-regime partitioning inside shared attention.[1]

Keywords: multi-tenant neural networks, cryptographic isolation, dimension partitioning, gate masks, Hadamard gating, formal verification, Lean 4, support-constrained training, permutation equivariance, classification, submodel extraction

1 Introduction

An identity infrastructure for model capability. X.509-style public-key infrastructure answered a precise question: *whose key is this, and who vouches for it?* A certificate binding a key to a name lets signed authority traverse organizations without shared custody. As trained models move from vendor servers into customer laptops, partner runtimes, and regulated estates, the analogous question about model artifacts becomes operational: *of the capability inside this artifact, which scope is this party entitled to execute, and whose signature says so?* Answering it requires more than authenticating a channel or encrypting a file: it requires identity-bound structure inside the artifact that signed grants can address the way certificates address keys — the internal capability surface of a public-key infrastructure for model capability. Cryptographic Dimension Partitioning (CDP) is developed here as that surface.

The economics of large language model (LLM) deployment push strongly toward sharing. A single 7-billion-parameter model requires roughly 14 GB for float16 weights or 7 GB for int8 weights, before KV caches, activations, allocator overhead, and serving state. Serving N tenants on N separate model copies scales linearly in cost. Parameter-efficient fine-tuning methods such as LoRA [Hu et al., 2022] offer an alternative: train a small adapter per tenant while sharing the backbone. Platforms like Predibase [Predibase, 2024] and S-LoRA [Sheng et al., 2024] serve hundreds of adapters from a single GPU, promising multi-tenant efficiency with per-tenant customization.

LoRA adds a low-rank perturbation $\mathbf{BA}$ to selected weight matrices while the base weights are commonly frozen. Its multi-tenant serving boundary is the adapter router and the associated batch, cache, and lifecycle state. A correctly routed adapter changes the hidden representation of its own request; that request-local diffusion is not, by itself, cross-tenant leakage. Another tenant is exposed only if routing, batching, cache namespacing, shared training, or another serving boundary admits the wrong adapter state. LoRA therefore provides efficient private state but does not inherently supply an IdP-bound cryptographic authorization gate or a coordinate-zero invariant.

Common LoRA configurations target attention projections ($\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$, $\mathbf{W}_O$), FFN layers, or both. Inside one active request, a dense shared attention head cannot be subdivided into independently normalized coordinate channels merely by assigning subsets inside its dot product: the partial sums are combined into one score before the shared softmax. This is a limitation on intra-attention coordinate partitioning, not a claim that a properly routed LoRA adapter is visible to every tenant.

Proposition 1 (Dense within-head attention coupling)**.** *Let $\mathbf{W}'_Q = \mathbf{W}_Q + \mathbf{B}_Q\mathbf{A}_Q$ be a LoRA modification to the query projection, where $\mathbf{B}_Q \in \mathbb{R}^{d\times r}$, $\mathbf{A}_Q \in \mathbb{R}^{r\times d_k}$. For any partition $\{1,\dots,d_k\} = S_i \sqcup S_j$, the modified attention score algebraically decomposes as*

$$(\mathbf{Q}'\mathbf{K}^\top)_{p,q} = \sum_{m=1}^{d_k} Q'_{p,m}K_{q,m} = \underbrace{\sum_{m\in S_i} Q'_{p,m}K_{q,m}}_{\text{subset } i} + \underbrace{\sum_{m\in S_j} Q'_{p,m}K_{q,m}}_{\text{subset } j}$$

[1] Code, artifacts, and proofs: https://github.com/sekosai/schemen-gate/tree/main/research/cdp

but this arithmetic decomposition does not produce independently owned channels. A generic dense update $\mathbf{B}_Q\mathbf{A}_Q$ is not support-constrained and may alter $Q'_{p,m}$ on either subset. Even if an update is constrained to one subset, the two partial scores are added before a single shared softmax; after that addition, a post-hoc coordinate mask cannot recover separately normalized tenant scores. Thus coordinate partitioning alone does not isolate tenants within one dense shared head. The proposition does not rule out a different architecture that computes complete attention and residual lanes separately and gates whole lanes before any recombination.

CDP has two architecture scopes. For partitioning within one model, the gate acts at an FFN boundary, where element-wise masking permits exact coordinate exclusion at the declared tensor. A *pre-MLP* gate masks the FFN input before $\mathbf{W}_1$; it is still an FFN placement, but its aligned parameter surface is limited to the input-facing rows of $\mathbf{W}_1$. An *intermediate-FFN* gate masks the expanded activation before $\mathbf{W}_2$ and confines the corresponding $\mathbf{W}_1$, $\mathbf{b}_1$, and $\mathbf{W}_2$ slices under a coordinate-respecting optimizer. The latter is the strongest formally and empirically evaluated placement in this paper. An extended attention embodiment is possible, but it is not a coordinate mask inside an ordinary shared head: it duplicates each Transformer block into regime-specific lanes, including each lane's complete set of heads and every residual path, and gates the whole lane before cross-lane recombination. That construction escapes the within-head premise of the proposition, but its duplicated topology creates a materially larger operator-error surface. Unless every attention and residual path is lane-aligned, attention remains inside the shared trust boundary. For the core FFN gate, $\mathbf{h} = \sigma(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)$; each hidden dimension h_j is computed independently. Zeroing h_j via the gate mask eliminates dimension j's contribution to the output exactly. No other hidden coordinate is changed by the multiplication itself; the subsequent $\mathbf{W}_2$ may mix the remaining active contributions across output coordinates.

For whole-model authorization, the complete model is instead one atomic resource behind a fail-closed runtime decision. The authorized path does not ablate or repartition internal activations. These FFN and whole-model scopes can be nested, but their guarantees are different.

Within either FFN placement, the fundamental mechanism is multiplication by a binary mask: $\mathbf{g}_r \in \{0,1\}^d$ is a vector of zeros and ones, and the gated activation is $\tilde{\mathbf{h}} = \mathbf{h} \odot \mathbf{g}_r$. Ones pass signal through; zeros annihilate it. This is the local confinement mechanism; authenticated dispatch and closure of other paths define the larger deployment boundary.

The *security layer* determines how masks are assigned and who may activate them. A lockbox-held assignment key drives an HMAC-SHA256 stream and unbiased Fisher–Yates shuffle, producing one global partition of the hidden dimension into R disjoint masks. The lockbox then issues scoped authorization material for an assigned mask; independently shuffling under each child key would not preserve global disjointness and is not the protocol. The authenticated runtime resolves a principal and operation to the assigned mask, so callers cannot choose a projection directly.

The resolved regime has a dual security role. Its selection is an authorization decision at the ingress boundary, while the resulting cryptographically bound regime is the authenticated execution identity for each downstream gate that verifies the same grant, model, operation, and policy scope. Thus the chain is external authentication, boundary authorization, then downstream regime authentication. A bare regime integer or activation tensor does not authenticate itself.

Contributions.

1. A capability-PKI formulation for model artifacts: identity-bound internal structure (partitions) that signed grants can address, with lockbox-generated assignment, so scoped capability can be conferred, composed, and revoked across parties without shared custody of the weights.
2. The CDP mechanism: a lockbox-generated global assignment from HMAC-SHA256 and unbiased Fisher–Yates to disjoint FFN masks, plus tenant + sub-id scoped authorization for FFN activation or atomic whole-model invocation.

3. Hadamard governance: one algebraic rule ($\tilde{\mathbf{h}} = \mathbf{h} \odot \mathbf{g}_r$) from which forward coordinate exclusion, active preservation, gradient isolation, and aligned FFN weight confinement follow.
4. Formal verification in Lean 4: the checked-in proof inventory contains 21 modules and 312 active theorem/lemma declarations with no active `sorry` or `admit` declarations. Five project-specific `axiom` declarations remain: two opaque predicates, one conditional PRF enumeration bound, and two historical statistical consequences. The core Hadamard and aligned FFN confinement results do not depend on them. The two statistical declarations (`camouflage_indistinguishable`, `gradient_probing_hard`) are excluded from the paper's claim set because their conclusions are not provable in the white-box setting; the declarations remain inspectable as historical conditional modules. The deployments they served are secured by the custody layer instead (see §6).
5. A placement theorem and counterexample identifying the safe *activation* → *gate* → $\mathbf{W}_2$ ordering and showing why pre-gate LayerNorm invalidates $\mathbf{W}_1$ confinement. The narrower pre-MLP placement is specified separately so its input-coordinate and first-projection guarantee is not confused with the intermediate theorem.
6. A strict three-seed DistilBERT cotenancy study that freezes all non-governed shared paths and measures intermediate-FFN, complete private-adapter, and private-expert designs. All unauthorized parameter and optimizer-state deltas are exactly zero; the dense FFN capacity curve is measured through R=16.
7. A Cargo Mode architecture and self-contained strengthened protocol runner binding tenant, sub-identity, regime, model digest, and operation before customer context can reach a shared frozen Transformer, plus a separate whole-model gate before generator invocation.
8. Supplementary support-constrained classification and keyed permutation-conjugation studies, explicitly separated from the intermediate-FFN isolation claim.

2 Related Work

Parameter-efficient fine-tuning. LoRA [Hu et al., 2022] introduces low-rank adapters on attention projections. QLoRA [Dettmers et al., 2024] extends this with 4-bit base models. Multi-tenant serving systems (S-LoRA [Sheng et al., 2024], Punica [Chen et al., 2023], LoRAX [Predibase, 2024]) batch multiple adapters on shared GPUs. These systems optimize adapter serving rather than providing a cryptographically authorized FFN-coordinate boundary; LoRA adapters are commonly installed on shared attention projections.

Privacy in fine-tuning. Differentially Private Stochastic Gradient Descent (DP-SGD) [Abadi et al., 2016] provides (ε, δ)-differential privacy bounds on the influence of training examples. It does not, by itself, route requests by tenant identity or allocate disjoint tenant-scoped model state in a serving process. Wang and Li [2023] study federated LoRA with DP guarantees. These approaches provide statistical privacy bounds; CDP provides exact local coordinate confinement at its declared gate (a different guarantee class).

Model partitioning. Expert parallelism in Mixture-of-Experts (MoE) [Fedus et al., 2022, Lepikhin et al., 2021] routes tokens to different FFN experts. The routing is learned and data-dependent (including sparse top-k routing); CDP's activation is instead resolved from an authenticated capability and a fixed assignment. The goals differ: MoE targets capacity scaling; CDP targets authorized FFN-coordinate confinement.

Task-specific masks and subnetworks. PackNet [Mallya and Lazebnik, 2018] allocates parameters to sequential tasks through iterative pruning; Piggyback [Mallya et al., 2018] learns binary weight masks over a fixed network; and Supermasks in Superposition [Wortsman et al.,

2020] retrieves task-specific subnetworks from a shared fixed base. These works establish that masks and disjoint or reusable subnetworks can support multiple tasks. CDP does not claim binary masking itself as novel. Its narrower contribution is the combination of a globally disjoint FFN activation assignment, exact aligned optimizer-state confinement, formal placement conditions, and identity-bound runtime activation.

Confidential and verifiable inference. Slalom [Tramèr and Boneh, 2018] partitions neural-network execution between trusted hardware and an untrusted accelerator to provide privacy and integrity for outsourced inference. CDP assumes rather than replaces a trusted custody and runtime boundary; its theorem addresses coordinate support inside that boundary, not confidentiality against a compromised host or accelerator.

Formal verification of ML systems. Seshia et al. [2018] survey formal methods for ML. Most work targets input-output robustness (e.g., certified adversarial bounds [Cohen et al., 2019]). CDP's Lean 4 proofs verify an *architectural* property (gradient and weight confinement) rather than an input-output property.

Representation superposition. Elhage et al. [2022] demonstrate that neural networks encode more features than they have dimensions via nearly-orthogonal directions. CDP makes no claim about that microscopic geometry; its support constraint is simply that excluded coordinates contribute zero, so each regime's usable gated contribution must be represented on its active support.

Permutation equivariance. The equivariance of linear algebra under change of basis is elementary. Our supplementary study applies a keyed residual-stream permutation consistently to a DistilBERT implementation and validates function preservation up to R=1,024. We do not claim the underlying permutation reparameterization as a new algebraic result.

3 Cryptographic Dimension Partitioning

3.1 Gate Mask Derivation

Definition 1 (Gate mask partition)**.** *Given an assignment key* $k_{\text{assign}} \in \{0,1\}^{256}$*, a number of regimes* $R \in \mathbb{N}$*, and a hidden dimension* $d \in \mathbb{N}$ *with* $d \geq R$*, a* gate mask partition *is a set of binary vectors* $\{\mathbf{g}_0, \ldots, \mathbf{g}_{R-1}\} \subset \{0,1\}^d$ *satisfying:*

$$\mathbf{g}_r \odot \mathbf{g}_s = \mathbf{0} \quad \forall\, r \neq s, \qquad \sum_{r=0}^{R-1} \mathbf{g}_r = \mathbf{1} \tag{1}$$

where $\odot$ *denotes the Hadamard (element-wise) product. The masks are disjoint and exhaustive: every dimension is owned by exactly one regime. Equal-sized regime partitions additionally require* $R \mid d$*, equivalently* $d = qR$ *for some integer* $q \geq 1$*; otherwise, regime sizes differ by at most one. The equal-sized construction and capacity formulas used below assume* $R \mid d$ *and write* $d_r := d/R$ *for the dimensions owned by one regime.*

The rejection threshold removes the incomplete residue tail before the modulo operation; `rejection_sampling_count` proves the corresponding equal-cardinality fact in Lean 4. Under the standard HMAC pseudorandom-function (PRF) assumption, the accepted words are computationally indistinguishable from uniform words. Rejection sampling prevents the modulo operation from introducing an additional residue imbalance into that pseudorandom stream. The result is deterministic for the same assignment key and encoded context. Mask secrecy is defense in depth: authorization ultimately depends on authenticated dispatch, not on treating a guessed coordinate set as a credential.

Algorithm 1 Global gate-partition assignment with unbiased draws

Require: Lockbox assignment key $k_{\text{assign}} \in \{0,1\}^{256}$, model digest m, version v, number of regimes R, hidden dimension d, with $d \geq R$ and $R \mid d$
Ensure: R disjoint binary masks $\{\mathbf{g}_0, \ldots, \mathbf{g}_{R-1}\}$ satisfying (1)

```
Initialize index array π = [0, 1, ..., d − 1]
for i = d − 1 down to 1 do                                   ▷ Fisher-Yates shuffle
    a ← 0; b ← i + 1; L ← 2^256 − (2^256 mod b)
    repeat
        u ← OS2IP(HMAC-SHA256(k_assign, Encode(partition-v1, m, v, R, d, i, a)))
        a ← a + 1
    until u < L                                              ▷ Reject the incomplete residue tail
    j ← u mod b
    Swap π[i] ↔ π[j]
end for
Partition π into R contiguous blocks of size d_r = d/R
for r = 0 to R − 1 do
    g_r ∈ {0,1}^d with g_{r,j} = 1 iff j ∈ block_r(π)
end for
```

3.2 Key Hierarchy and Token System

CDP domain-separates global assignment from authorization. The lockbox derives one assignment key for the complete model partition and separate scoped capability keys:

$$k_{\text{assign}} = \text{HKDF-Expand}(k_{\text{root}}, \texttt{assignment} \| m \| v \| R \| d), \tag{2}$$

$$k_{\text{scope}} = \text{HKDF-Expand}(k_{\text{root}}, \texttt{authorization} \| \mathcal{S}), \tag{3}$$

where the canonical scope tuple is

$$\begin{aligned} \mathcal{S} = (&\text{issuer}, \text{subject/sub-id}, \text{tenant}, \text{regime/capability}, m, \\ &\text{resource IDs}, \text{operation}, \text{grant ID}, \text{key epoch}, \\ &\text{expiry}, \text{policy version}). \end{aligned} \tag{4}$$

The assignment is generated once from k_{assign} and stored by identifier. A child capability does not independently shuffle the model; it authorizes an assignment already present in the global partition. Tokens may encrypt that assignment identifier and capability payload with AES-256-GCM while binding the canonical scope as Associated Authenticated Data (AAD). Tampering with a bound field causes authentication failure.

Tenants do not choose or execute the Fisher–Yates shuffle. The lockbox controls derivation authority, and the trusted runtime resolves an authenticated credential to a scoped capability and its assigned mask. A caller presenting a credential for regime s cannot ask the runtime to apply regime r's assignment. Possessing or guessing mask indices is not sufficient because the runtime does not accept caller-supplied masks.

AAD authenticates claims; it does not evaluate them. Immutable fields such as tenant, sub-id, regime, model digest, operation, and policy version are checked against the request during decryption. A grant may additionally authenticate administrative fields such as a parent grant, allowed operations, an input/output-token ceiling, a maximum invocation count, or a delegation right. Dynamic requirements such as a remaining-use counter, geography, current account state, or revocation status must be evaluated by the policy runtime using those authenticated claims. In particular, placing `max_uses=N` in AAD makes the limit tamper-evident but does not decrement it. Exact-use enforcement requires an online atomic ledger that reserves a use before invocation,

rejects replay, and fails closed when current state cannot be read. The capability is therefore the combination of an authenticated scope, lockbox grant, current policy state, and enforcing runtime—not the mask vector alone.

3.3 Gate Surfaces

The word *gate* denotes two related enforcement surfaces, not one claim stretched across every architecture. Within an FFN, the gate is a Hadamard mask on a declared activation. Around a complete model, it is a fail-closed execution decision. The location determines the state covered by the guarantee.

Pre-MLP gate (FFN input). Let $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ be the input to an MLP/FFN. A pre-MLP placement computes

$$\bar{\mathbf{x}} = \mathbf{x} \odot \mathbf{g}_r, \qquad \mathbf{h} = \sigma(\bar{\mathbf{x}}\mathbf{W}_1 + \mathbf{b}_1). \tag{5}$$

This is an FFN gate even though it precedes the first linear projection. Excluded input coordinates, their loss gradients, and the loss gradients of the aligned input-facing rows of $\mathbf{W}_1$ are zero. This placement alone does not partition the expanded hidden activation, $\mathbf{W}_2$, or parameters upstream of a mixing operation.

Intermediate-FFN gate. The stronger parameter-aligned placement computes

$$\mathbf{h} = \sigma(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1), \qquad \tilde{\mathbf{h}} = \mathbf{h} \odot \mathbf{g}_r, \qquad \hat{\mathbf{y}} = \tilde{\mathbf{h}}\mathbf{W}_2 + \mathbf{b}_2. \tag{6}$$

It zeros excluded hidden coordinates and aligns the loss-gradient boundary with columns of $\mathbf{W}_1$, entries of $\mathbf{b}_1$, and rows of $\mathbf{W}_2$. The formal weight-confinement theorem and strict DistilBERT experiment use this placement.

Whole-model gate. For model resource M and operation o, let $a(c, M, o) \in \{0, 1\}$ be the trusted runtime's authorization decision after resolving the authenticated tenant + sub-id scope. The atomic execution gate is

$$\text{Exec}(c, M, \mathbf{x}, o) = \begin{cases} M(\mathbf{x}), & a(c, M, o) = 1, \\ \perp, & a(c, M, o) = 0, \end{cases} \tag{7}$$

where $\perp$ means rejection before model invocation. No internal coordinate is removed on the authorized path, so this gate introduces no model-capacity loss; it relies on the IdP, lockbox/proxy, runtime, and serving-state boundary. A model bank may contain many atomic model resources, but that resource count is not the FFN partition ratio d/R. Whole-model authorization therefore does not claim multi-regime separation inside one shared Transformer. The decision is nevertheless a structured capability rather than a bare Boolean: it can bind the model version, operation, tenant + sub-id, expiry, delegation chain, token ceiling, use budget, and audit identifier. Several atomic gates may be chained around a pipeline—for example, retrieval, reranking, generation, and a tool call—with each stage requiring its own resource-and-operation grant. The authorized model computation remains unchanged at every stage.

3.4 Hadamard Governance

The FFN gate is governed by exactly one algebraic rule:

Definition 2 (Hadamard gate)**.** *For hidden activation* $\mathbf{h} \in \mathbb{R}^d$ *and binary mask* $\mathbf{g}_r \in \{0, 1\}^d$*:*

$$\tilde{\mathbf{h}} = \mathbf{h} \odot \mathbf{g}_r \tag{8}$$

Table 1: The four governance properties of Hadamard gating. Each is a one-line proof in Lean 4 (all reduce to `simp`).

Property	**Statement**
Forward isolation	$g_{r,j} = 0 \implies (\mathbf{h} \odot \mathbf{g}_r)_j = 0$
Active preservation	$g_{r,j} = 1 \implies (\mathbf{h} \odot \mathbf{g}_r)_j = h_j$
Gradient isolation	$g_{r,j} = 0 \implies (\partial \mathcal{L} / \partial h_j) = 0$
Gradient confinement	$(\partial \mathcal{L} / \partial h_j) \neq 0 \implies g_{r,j} = 1$

From this single definition, four governance properties follow, each a direct consequence of $0 \cdot x = 0$ and $1 \cdot x = x$ propagated through the chain rule:

The IdP-authenticated runtime decides which mask is authorized; the Hadamard product enforces that resolved decision at the declared tensor. The runtime is therefore essential to authorization, while $0 \cdot x = 0$ supplies the local coordinate exclusion and gradient exclusion once the correct mask is applied.

Computational overhead of the gate. The gate is one element-wise multiply over d coordinates per gated tensor per invocation: d scalar multiplications with no accumulation in the forward pass and one broadcast multiply in the backward pass, against the $\Theta(d \cdot d_{\text{in}})$ and $\Theta(d \cdot d_{\text{out}})$ MACs of the two projections it guards. It is emphatically not an extra matrix multiplication: implemented as a dense — or even an explicit diagonal — matrix product $\text{diag}(\mathbf{g}_r)\,\mathbf{h}$ it would cost $\Theta(d^2)$ MACs per gated boundary, but that is an implementation error, not a property of the mechanism. Evaluated as element-wise broadcast multiply (as in the checked-in implementations) its memory traffic is two vector reads and one vector write and may be fused into the epilogue or prologue of a surrounding GEMM. For inference-output equivalence, a mask that is constant for a packaged artifact may instead be folded into the aligned rows of $\mathbf{W}_2$. That reparameterization removes the per-request multiply but does not instantiate the declared activation-level zero or the training-state confinement theorem; it must be described as extraction or packaging, not as the same gate surface. A bit-packed mask requires d bits (96 bytes at d=768), while an expanded Boolean or floating-point runtime tensor requires implementation-dependent additional storage. Enforcement is therefore not free, but the unfused operation is linear in the gated width rather than quadratic. We make no numerical latency claim for the gate: the only systems figures in this repository are Section 8.7's state-dict arithmetic and Section 9.1's keyed-basis benchmark, the historical throughput figures are withheld pending hardware-matched rerun (Section 8.7), and no dedicated gate-overhead microbenchmark is yet part of the checked-in evidence.

Orthogonality of disjoint gated activations. For two regimes r, s with disjoint masks ($\mathbf{g}_r \odot \mathbf{g}_s = \mathbf{0}$), the gated activation vectors have zero inner product:

$$\langle \mathbf{h} \odot \mathbf{g}_r,\ \mathbf{h} \odot \mathbf{g}_s \rangle = \sum_{j=1}^{d} h_j^2\, g_{r,j}\, g_{s,j} = 0 \tag{9}$$

because $g_{r,j} g_{s,j} = 0$ for every coordinate. This is an algebraic support-orthogonality statement, not a claim that the random variables represented by different regimes are statistically independent or that shared model paths carry no correlated information.

3.5 Gated Forward Pass

Definition 3 (Element-wise FFN independence)**.** *In a standard transformer FFN, each hidden dimension is computed independently:* $h_j = \sigma(\mathbf{x} \cdot \mathbf{W}_1[:, j] + b_{1,j})$. *The output contribution of*

dimension j is $\hat{y}_m = \sum_j \tilde{h}_j W_{2,j,m} + b_{2,m}$. Zeroing $\tilde{h}_j$ via the gate eliminates dimension j's contribution to every output element exactly.

For input $\mathbf{x}$, regime r with gate mask $\mathbf{g}_r$:

$$\mathbf{h} = \sigma(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1) \tag{10}$$

$$\tilde{\mathbf{h}} = \mathbf{h} \odot \mathbf{g}_r \tag{11}$$

$$\hat{\mathbf{y}} = \tilde{\mathbf{h}}\,\mathbf{W}_2 + \mathbf{b}_2 \tag{12}$$

The output depends only on the $d_r = d/R$ dimensions owned by regime r. This is the architectural reason CDP uses FFN dimensions as its core partition: they are independent, whereas dimensions inside a dense shared attention head are coupled through $\mathbf{QK}^\top$. Attention isolation requires the separate lane construction described in Section 5, not a post-hoc coordinate mask inside one head.

3.6 Gradient and Weight Confinement

During backpropagation, the chain rule propagates the gate mask into the gradient:

$$\frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \mathbf{W}_2^\top \tag{13}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}} \odot \mathbf{g}_r \tag{14}$$

For any dimension $j \notin \mathrm{supp}(\mathbf{g}_r)$: $g_{r,j} = 0 \implies \partial \mathcal{L}/\partial h_j = 0$. This is exact, not approximate, and holds unconditionally for all losses, batch sizes, and inputs at the gradient level. Consequently:

- $\mathbf{W}_1[:, j]$ receives zero loss gradient for $j \notin \mathrm{supp}(\mathbf{g}_r)$ (proven: `weight_update_confined`). The corresponding hidden bias $b_{1,j}$ receives the same masked preactivation gradient.
- $\mathbf{W}_2[j, :]$ receives zero loss gradient for $j \notin \mathrm{supp}(\mathbf{g}_r)$ because $\tilde{h}_j = 0$. This is `w2_update_confined`.

The output bias $\mathbf{b}_2$ is not coordinate-indexed by the hidden partition and is therefore shared unless frozen, separately scoped per regime, or gated by an additional declared mechanism. It is not covered by the $\mathbf{W}_1/\mathbf{W}_2$ confinement theorem.

Exact parameter-state confinement additionally requires the optimizer to respect the same coordinate boundary: decoupled weight decay, stale momentum, and other stateful update terms must be masked per coordinate or maintained per regime. Subject to that optimizer condition, updates to the gated FFN parameters are confined to the regime's coordinate partition. The gate restricts both what the output can source (forward) and where loss gradients can flow (backward); shared components outside this surface are identified in Section 5.1.

3.7 Regime Destruction (Algebraic Annihilation)

Destroying regime r consists of zeroing the weight columns and rows corresponding to $\mathrm{supp}(\mathbf{g}_r)$:

$$\mathbf{W}_1[:, j] \leftarrow \mathbf{0}, \quad b_{1,j} \leftarrow 0, \quad \mathbf{W}_2[j, :] \leftarrow \mathbf{0} \quad \forall\, j \in \mathrm{supp}(\mathbf{g}_r) \tag{15}$$

By the confinement theorems, no regime-r update exists outside $\mathrm{supp}(\mathbf{g}_r)$ within the confined FFN parameter surface. After zeroing, those parameters are identical to parameters that never received regime-r loss gradients. If $\mathbf{b}_2$, all other tenant-trainable paths, and optimizer state are frozen, separately partitioned, or erased under the same boundary, the statement extends to the complete declared regime-scoped trainable state. We call this algebraic annihilation: mathematical zeroing of the declared parameter slice. It does not erase shared pretrained representations, logs, caches, or backups.

3.8 Cross-Mask Additive Cancellation

A narrow adversarial question is whether an additive vector confined to one mask can appear through a disjoint mask at the same gate. The answer is no under the theorem below.

Theorem 2 (Injection confinement)**.** *Let $\mathbf{z} \in \mathbb{R}^d$ be a knowledge vector injected additively into the residual stream, confined to regime r's coordinate subspace via the gate mask. For any other regime $s \neq r$, the gated output of regime s is identical with and without the injection:*

$$(\mathbf{h} + \mathbf{z} \odot \mathbf{g}_r) \odot \mathbf{g}_s = \mathbf{h} \odot \mathbf{g}_s$$

Proven in Lean 4: `injection_confined_vec`*.*

The proof follows from two facts: (1) $\mathbf{z} \odot \mathbf{g}_r$ is zero at every dimension outside $\text{supp}(\mathbf{g}_r)$, and (2) $\text{supp}(\mathbf{g}_r) \cap \text{supp}(\mathbf{g}_s) = \emptyset$ for $r \neq s$. The injection vanishes at every dimension regime s owns at that gate. This does not cover information routed through shared components or ungated bypasses.

Injection confinement complements aligned FFN gradient confinement: one constrains an explicitly gated additive vector, while the other constrains updates on the declared FFN parameter surface. Neither statement extends to shared or bypass paths.

3.9 Mask Algebra and Authorized Union

Binary masks are closed under AND, NOT, and OR. This algebraic fact does not make every operation an authorized runtime primitive:

Definition 4 (Regime composition)**.** *For regimes r, s with masks $\mathbf{g}_r, \mathbf{g}_s$:*

$$\mathbf{g}_{r \cap s} = \mathbf{g}_r \odot \mathbf{g}_s \quad \textit{(AND: element-wise product)} \tag{16}$$

$$\overline{\mathbf{g}}_r = \mathbf{1} - \mathbf{g}_r \quad \textit{(NOT: element-wise complement)} \tag{17}$$

$$\mathbf{g}_{r \cup s} = \overline{\overline{\mathbf{g}}_r \odot \overline{\mathbf{g}}_s} \quad \textit{(Union: derived via De Morgan)} \tag{18}$$

AND and NOT are sufficient for functional completeness; Union is a convenience derived as $\mathbf{g}_{r \cup s} = \mathbf{1} - (\mathbf{1} - \mathbf{g}_r) \odot (\mathbf{1} - \mathbf{g}_s)$. For disjoint regimes, AND returns the zero vector and Union returns the concatenation of both supports.

The production-facing primitive considered here is an explicit **authorized union**, which activates both supports in one forward pass. The Lean 4 declarations `compose_disjoint` and `compose_access` establish the mask-level facts that each constituent support remains active and coordinates outside the union remain excluded. They do not prove end-to-end safety of shared model components.

The complement $\overline{\mathbf{g}}_r$ activates every coordinate outside r and therefore cannot be inferred as an authorization grant from possession of r alone. A runtime may construct such a mask only if the resolved scope independently authorizes every activated support.

Union is an authorized operation: the caller must possess a valid grant for every constituent regime. Those grants may belong to different tenants who consent to composition, or to one principal that is intentionally assigned several capabilities (for example, multiple MoE experts). Under the trusted-runtime assumptions, the runtime refuses to construct the Union mask without all required grants. A multi-regime grant deliberately removes the FFN-coordinate boundary among its members while preserving mask-level exclusion for coordinates outside the union.

Whole-model wrapping. A complete model may be registered as one atomic resource behind (7), regardless of how many other models the runtime serves. This is not a degenerate FFN mode: the enforcement event is authorization before invocation, not multiplication of every internal activation by an all-ones mask. If an inner FFN partition is also configured with R_{FFN}=1, its mask $\mathbf{g}_0 = \mathbf{1}$ is a computational no-op, but the outer model gate still supplies identity, operation, lifecycle, delegation, metering, and audit control. It can also serve as one link in a capability chain: authorization to invoke model M_1 need not imply authorization to invoke a downstream model M_2, retrieve corpus D, or call tool T. Each edge is resolved from its own AAD-bound resource and operation scope. Later internal partitioning requires separate gate-aware validation and may change utility; atomic whole-model authorization does not predict $R_{\text{FFN}} > 1$ behavior.

Nested composition (CDP inside CDP). An outer whole-model gate and an inner FFN gate compose sequentially: the runtime first authorizes model M, then M executes with the authorized inner mask $\mathbf{g}_r$. The outer decision acts on a resource identifier; the inner mask acts on an activation tensor. They are not one Hadamard vector, and the Lean 4 coordinate proofs cover the inner FFN gate rather than the outer control-plane decision.

When multiple masks are defined over the same indexed activation space, associativity permits several masks to reduce to one binary vector. If outer and inner policies act on different axes—for example expert identity and coordinates inside an expert—they must first be lifted into one product space or a block-diagonal mask. Their original vectors cannot be multiplied directly.

This nested structure is particularly natural for Mixture-of-Experts (MoE) architectures, where each expert is already a separate FFN block. A CDP partition at the expert level can provide *capability selection* (which experts a principal can activate), while a CDP partition within each expert provides *coordinate confinement* (which dimensions inside an expert are active). Under the product-space lifting above, the two levels compose because disjointness is preserved under Hadamard product: if $\mathbf{g}_i^{\text{outer}} \odot \mathbf{g}_j^{\text{outer}} = \mathbf{0}$ and $\mathbf{g}_r^{\text{inner}} \odot \mathbf{g}_s^{\text{inner}} = \mathbf{0}$, then the composed masks are also disjoint.

The expert-routing plane. Between the two levels sits an expert-router authorization surface. A MoE layer selects experts through a routing projection that produces expert logits and dispatch weights. Enforcement must respect that router's order of operations. One conforming construction first restricts candidates to the authorized set A (equivalently, sets every unauthorized logit to $-\infty$), selects at most $k' = \min(k, |A|)$ finite entries, and refuses to dispatch any $-\infty$ entry. Another multiplies normalized dispatch weights by a grant-scoped binary mask before dispatch and removes every zero-weight entry. Both must fail closed when A is empty. Plain multiplication of pre-softmax logits by zero is insufficient because softmax can assign positive mass to a zero logit; setting logits to $-\infty$ without constraining a fixed-size top-k is likewise insufficient when $|A| < k$. Exact zero contribution follows algebraically when normalized dispatch weights are masked; exclusion from selection and computation additionally depends on the fail-closed router implementation. The distinction to keep straight is one of control versus routing: MoE routing is data-dependent and learned, deciding *which tokens* see which experts; CDP authorization is data-independent and grant-scoped, deciding *which principals* may reach which experts at all.

We evaluate this composition on eight disjoint binary tasks from the standard 20 Newsgroups train/test split. Each regime independently trains a top-1 router over two private feed-forward experts with task loss and a load-balancing loss computed only inside its authorized set. The eight trained routers and sixteen experts are copied unchanged into one execution-authorized bank. Across 6,231 held-out examples, macro accuracy is 81.80% and micro accuracy is 81.86% (5,101 correct). Separate and packed execution have exactly equal logits and predictions; unauthorized expert dispatch is zero. Both experts are used in every regime, with the least-used expert receiving 42.4% of its regime's held-out examples, and every router records nonzero parameter movement. A dedicated active-regime training-step audit records zero inactive gradient tensors,

optimizer states, and parameter change. Wrong regime, model, dimension, and empty-authority probes against eight single-regime execution surfaces make zero model calls.

The deliberately invalid control multiplies unauthorized pre-softmax logits by zero; an unauthorized expert wins top-1 because zero exceeds the authorized negative logits. Candidate restriction or $-\infty$ masking before softmax is therefore an empirically exercised placement requirement. Exact zero dip here is a copy-and-pack equivalence result. The packed bank has the same router/expert bytes as the separate lanes, so this experiment does not itself claim storage compression, shared-attention isolation, or language-model quality.

Replay learned routing. Run `experiments/train_authorized_moe.py`; the clean-source artifact is `experiments/results/authorized_learned_moe_20260831T182431_055309Z.json`.

Execution-selected capability prefixes and token routing. Learned continuous prefixes can condition model computation without changing the user-visible token sequence [Li and Liang, 2021]. They are not, by themselves, credentials. Any plaintext prefix accepted from a request is forgeable and remains untrusted data. Our construction therefore separates the channels: authenticated execution authority selects a private learned prefix and its allowed expert set; user tokens then condition top-1 routing only within that set. The prefix enters the token router as a private continuous context, not as user-authored text, and authorization is applied before softmax and dispatch.

We evaluate eight private prefixes and sixteen experts on the same held-out 20 Newsgroups task family. Across 311,477 non-padding token decisions, macro accuracy is 77.76% and micro accuracy is 77.80% (4,848 of 6,231 examples). Copying the eight independently trained token routers, prefixes, and experts into one authorized bank changes zero token routes, logits, or predictions. Global negative-infinity masking exactly matches candidate restriction; unauthorized dispatch is zero. A spoof request containing literal `CAPABILITY_REGIME_0` through `CAPABILITY_REGIME_7` as user text also causes zero unauthorized dispatch. Every learned prefix and router moves, both experts are used in every regime with a minimum held-out share of 41.7%, and an active-regime update creates zero inactive gradients, optimizer states, or parameter change. All malformed probes against eight singleton execution authorities make zero model calls.

This is a small token-routing classifier, not an autoregressive Transformer. The result does not establish prompt text as authority, language-model generation quality, shared-attention isolation, or storage compression. Exact zero dip is again the copy-and-pack equivalence; task utility is reported separately.

Replay capability-prefix routing. Run `experiments/train_capability_token_moe.py`; the clean-source artifact is `experiments/results/capability_prefix_token_moe_20260831T182453_013039Z.json`.

4 Threat Model and Trusted Execution

4.1 System and Assets

The system input is not a prompt $\mathbf{x}$ alone but a pair $(\mathbf{x}, c)$, where c is an authenticated user or workload credential. The IdP and issued lockbox grant resolve c to the canonical scope $\mathcal{S}(c)$ from (4); tenant identity without the authenticated subject/sub-id is incomplete. A principal may hold one regime or an authorized set of capabilities $S \subseteq \{0, \ldots, R-1\}$. The trusted control path resolves

$$c \longmapsto \mathcal{S}(c), \quad S, \quad \{(k_{\text{scope},r}, \mathbf{g}_r) : r \in S\}, \quad \mathbf{g}_S = \bigvee_{r \in S} \mathbf{g}_r \tag{19}$$

before model execution, where $\bigvee$ is element-wise binary OR. For a single-regime credential, $S = \{r\}$ and $\mathbf{g}_S = \mathbf{g}_r$. For a whole-model gate, the same canonical scope instead resolves a model resource M and operation o and requires $a(c, M, o) = 1$ before invocation; no FFN mask is

implied by that authorization. The union is defined only for regime masks expressed in the same active model basis. The residual-stream permutations P_r introduced in Section 9 are alternative model-wide keyed placements, not coordinates that may be ORed directly. A grant spanning different keyed basis placements must execute each placement separately and compose only at a declared output boundary; the current runtime does not claim single-pass union across distinct P_r bases. The protected assets are the root and assignment keys, lockbox authority, scoped capability keys, permutation assignments, masks, declared regime-scoped FFN coordinates, model attestations, and the audit record binding those objects to the complete canonical scope.

The intended trust chain is

$$\text{IdP} \rightarrow \text{lockbox/Vault} \rightarrow \text{sidecar} \rightarrow \text{runtime} \rightarrow \text{gate}.$$

The implementation is not coupled to a particular trusted-compute vendor. Its portable machine-credential path uses PKCS#12 to package an Ed25519 private key, leaf certificate, and certificate chain, while the verifier pins the accepted root independently. PKCS#12 is a credential container rather than evidence of TPM, enclave, measured-boot, or confidential-computing residency. The reference path loads the key into process memory; deployments requiring non-exportable keys must supply a native signing provider and separate platform evidence. A deliberately pinned self-signed root is also valid, but establishes only possession and continuity of that pinned key.

This chain contains two identities that must not be conflated. An OIDC IdP authenticates the human or calling principal and supplies the issuer and stable subject/sub-id. SPIFFE/SPIRE supplies short-lived, attested workload identities (SVIDs) and trust bundles to the lockbox, sidecar, and runtime workloads [SPIFFE Project, 2026]. SPIFFE is therefore the recommended service-to-service authentication layer; it does not replace the end-user IdP or decide which tenant resource the user may access.

The IdP-authenticated principal resolves to a tenant, resource, and operation scope at the lockbox or Vault-backed provider, which controls principal-scoped key derivation and grant issuance for the authorized regime or capability set. A sidecar must authenticate both the current grant and the workload allowed to redeem it, release only the authorized mask or invocation decision, and never accept a caller-selected permutation. The runtime is the leaf of this chain: it applies the resolved gate immediately at the declared surface and namespaces serving state under the same canonical scope.

For a metered whole-model grant, the strict request order is: validate the OIDC principal; authenticate the enforcing workload with an SVID; atomically reserve the grant's invocation and token budget; obtain the current mask or model-execution decision; invoke the declared resource; settle the measured token count; and append an audit event. SPIFFE authenticates the workload performing those steps. The atomic ledger, rather than the SVID or AAD alone, enforces an exact number of uses.

4.2 Adversary and Trust Assumptions

The in-scope adversary is an unauthorized tenant or network caller who can submit chosen inputs, observe released outputs, train on data within its own authorized regime set, and possess its own scoped credential or capability material. The adversary may substitute a tenant, sub-id, regime, model, resource, or operation field; tamper with a token or model volume; or attempt to compose regimes without the required grants. It does not control the IdP, lockbox, Vault, sidecar, trusted runtime process, or master gate seed.

Compromise of that trusted custody boundary, process-memory extraction after a mask is loaded, or an operator intentionally issuing an unauthorized grant is outside the cryptographic threat model. It is nevertheless an explicit system risk. The term *lockbox* covers both a sealed distributable artifact and the authority that holds the root, assignment, and signing material. Stealing a sealed artifact alone does not reveal recipient-wrapped keys. Exfiltrating the live

authority, root/assignment key, or an unrestricted derivation-and-signing capability is a keys-to-the-kingdom failure: the attacker can reconstruct assignments and mint apparently valid grants for every scope. Recovery requires disabling the old authority, rotating the root and accepted key epoch, reissuing grants, and, where assignment secrecy matters, regenerating and reattesting the partitioned model.

Key assignment is therefore an operator-controlled part of the trusted computing base: isolated scopes require distinct capability contexts, while key reuse or a multi-regime grant intentionally joins their capability boundary. Likewise, components declared shared in Table 2 are not silently promoted into the algebraic guarantee: KV-cache blocks, optimizer state, logs, and other stateful artifacts must be separately namespaced by the same resolved regime identity. Gate placement and path closure remain mandatory as described in Section 5.1.

4.3 Credential Binding and Fisher–Yates Custody

Callers never choose the Fisher–Yates permutation. The lockbox controls the global assignment and capability authority, only scoped material is released, and the runtime obtains the mask through the authenticated sidecar path. In the strict path, the OIDC validator resolves issuer and subject/sub-id, the grant binds tenant, regime/capability, model digest, resource, operation, grant ID, key epoch, and expiry, and SPIFFE authenticates the workload that requests or redeems the grant. The complete scope must match the request, the current grant, and the stored artifact before mask indices or an invocation decision are returned. A caller-supplied `regime_id` is not sufficient. Multi-capability execution requires an authenticated grant covering every member of S; the current single-operator path uses an administrator-authorized compound request, while strict cross-authority grants remain an integration seam. The sidecar separately verifies ownership before proxying a request. Vault ACLs and lockbox grants govern operator and workload access to derivation operations.

Deriving a separate key for each principal makes revocation granular, but derivation does not claw back bytes already disclosed. A strict grant therefore includes a unique grant ID and monotonically changing key epoch in its HKDF context. Revoking one principal marks that grant inactive and advances its accepted epoch; every mediated invocation checks current state before use. Other principals need not rotate. Removing a recipient from a newly issued lockbox, by itself, does not invalidate an old derived key copied for offline use. A deployment that accepts such keys without an online current-epoch check cannot claim immediate principal revocation.

Therefore guessing a permutation is not itself an authorization event. Dispatch integrity relies on the IdP, lockbox, sidecar, and runtime boundary; assignment secrecy under the HMAC-SHA256 PRF assumption is defense in depth. The public reproduction bundle contains the FFN gates, a minimal fail-closed Cargo scope harness, and an atomic whole-model invocation check. Production OIDC, Vault, and sidecar implementations are outside this repository and are not claimed as independently reproduced here.

4.4 Guarantees Under Correct Dispatch

For a whole-model scope, correct dispatch either invokes the complete, unchanged model or rejects before invocation. This is an operational authorization guarantee under the trusted runtime boundary, not an internal coordinate-separation theorem.

For an invocation resolved to capability set S in one active model basis, the runtime applies $\mathbf{g}_S$. For every coordinate owned by a regime $r \notin S$, disjointness makes the corresponding activation exactly zero at the correctly placed gate. The full activation need not be the zero vector: coordinates in the authorized union remain active. Thus the structural claim is non-access outside S. A single-capability invocation commonly uses $|S| = 1$; a capability or MoE deployment may intentionally use $|S| > 1$. In the latter case, sharing inside S is authorized behavior, not

cross-tenant leakage. Any wrong-key task-accuracy effect is downstream and empirical rather than part of the gate theorem.

Matched conjugation and wrong-regime execution are also distinct conditions. A complete conjugation P_r paired with its matching runtime basis preserves the base function exactly. A mismatched credential, basis, mask, head, or classifier leaves an uncancelled coordinate relabeling and does not reproduce the intended regime. Permutation conjugation provides the keyed placement; the Hadamard gate provides zeros on excluded FFN coordinates.

The operator is responsible for preserving the intended key cardinality. If two regimes that are claimed to be isolated are assigned the same effective key, mask, or unrestricted compound grant, CDP correctly enforces the resulting joined capability but cannot infer that the policy was mistaken. The runtime reduces this risk by deriving keys in regime-specific contexts, validating identity-bound grants, and refusing caller-selected permutations.

4.5 Prospective Knowledge Lifecycle and BYOK

A provider could reserve an unassigned FFN partition for later regime-scoped training. The confinement theorem predicts that a correctly placed intermediate FFN gate and coordinate-respecting optimizer would restrict parameter updates to that partition. However, the present experiments do not establish successful owning-key learning in a previously unused partition. Hollow-regime installation, tenant-corpus deposition, per-regime heads or adapters, and injected encoders are therefore prospective embodiments rather than evaluated contributions of this paper.

In the BYOK embodiment, the tenant additionally supplies an inner key that derives $\mathbf{g}^{\text{tenant}}$. The effective mask is

$$\mathbf{g}_r^{\text{effective}} = \mathbf{g}_r^{\text{regime}} \odot \mathbf{g}^{\text{tenant}}.$$

The platform can authorize the outer regime without possessing the tenant's inner key. At the gate, a coordinate is active only when both masks include it. Tenant-specific content utility under that construction is prospective. Nested-mask and portable PKCS#12 key provider paths exist as reference implementations; strict hardware-backed key custody and end-to-end BYOK integration with every production runtime path is an embodiment boundary, not an assumption of the core gate theorem.

4.6 Mandatory Enforcement Architecture

CDP is an architecture comprising the authenticated IdP route, lockbox or Vault-backed derivation, enforcing sidecar, trusted runtime, and correctly placed gate. A conforming deployment must resolve the authenticated canonical scope through that chain; an unverified request `tenant_id`, `sub_id`, or `regime_id` is not an authorization input. Development routes that accept direct identifiers or injected keys bypass CDP and cannot claim its enforcement properties.

Audited implementation status. The public release contains the Gate library and no serving or control-plane implementation. It provides deterministic mask derivation, scoped AEAD and HMAC contracts, recipient-wrapped grants, signed lockboxes, exact signed-grant membership checks, PKCS#12/X.509 verification against operator-configured roots, finite operation gates, and the fail-closed Cargo protocol. Cargo binds tenant, subject, Regime, model, operation, policy version, payload semantics, and exact partition; its success receipts require declared load completion and commit to material retrieval results.

Current experiments use a small source-visible callback preflight that checks model identity, dimensions, and Regime before invoking a model. This fixture is not an IdP, credential verifier, network service, durable use ledger, or production authorization layer. Accordingly, the repository makes no claim that an end-to-end production principal path is shipped.

The production profile requires an operator to authenticate the principal, resolve a signed canonical scope, distribute trusted roots, protect keys, enforce expiry and revocation, redeem

finite uses atomically across replicas, apply the Gate before every protected operation or declared FFN path, and close all alternate routes. Those deployment properties require separately pinned evidence from the actual integration.

5 The Trust Boundary

5.1 Isolation Surface by Embodiment

CDP's *isolation surface* is the set of activations, parameters, optimizer state, and serving state for which a deployment enforces a regime boundary. The surface is embodiment-specific; the binary gate always gives exact forward and loss-gradient confinement at the activation on which it acts. That local algebra does not certify an arbitrary installation point. For the declared surface to be isolated, the gate must precede the first operation that mixes protected coordinates into unprotected ones, and every path carrying the protected representation must cross an equivalent gate. A downstream or bypassed gate can still zero its immediate tensor while failing to isolate the computation as a whole.

Table 2: Isolation surface by gate placement. Pre-MLP and intermediate placements are both FFN gates but cover different parameter axes. Whole-model gating is an operational invocation boundary, not an internal coordinate partition.

Component or state	**Pre-MLP FFN**	**Intermediate FFN**	**Whole model, atomic**
Gated tensor	FFN input coordinates	Expanded hidden coordinates	No internal tensor mask
Aligned loss-gradient and parameter surface	Input gradient and $\mathbf{W}_1$ input-facing rows	$\mathbf{W}_1$ hidden columns, $\mathbf{b}_1$, and $\mathbf{W}_2$ rows	Complete model invoked or rejected as one resource
Attention, embeddings, normalization, output paths	Shared; outside local gate	Shared; outside local gate	Inside authorized atomic resource; not internally partitioned
Optimizer moments and decoupled weight decay	Mask aligned $\mathbf{W}_1$ rows	Mask aligned FFN slices	Scope complete model-training state if training is authorized
KV cache, adapters, extracted artifacts, logs	Execution-scoped	Execution-scoped	Model-scope and principal-scope runtime namespacing
Authorized-path capacity effect	Uses active FFN-input support	Uses active expanded-FFN support	None: authorized model computation is unchanged

The local learned-state claim follows the declared FFN placement. A pre-MLP gate covers input-facing $\mathbf{W}_1$ rows; the stronger evaluated intermediate gate covers aligned $\mathbf{W}_1$, $\mathbf{b}_1$, and $\mathbf{W}_2$ slices. Shared pretrained infrastructure may supply common computation to all regimes. If shared components are co-trained on regime data, their updates are outside either local FFN guarantee. Successful content-specific deposition is not established by the current experiments. Stateful artifacts such as optimizer moments and KV-cache blocks are not partitioned by multiplication alone and must follow the same runtime-resolved regime identity.

Whole-model authorization. The entire model—attention, embeddings, FFNs, output projection, normalization, and associated serving state—is registered as one atomic resource. Isolation is operational: the key hierarchy, IdP-bound policy, runtime, audit, state namespacing, and lifecycle controls must reject unauthorized invocation and state access. A runtime may serve many such model resources, so this mode is not defined by R_{FFN}=1. Nor does it separate two principals who are intentionally authorized to the same atomic model. Its value is the structured authorization envelope around unchanged computation: tenant + sub-id, model/version, operation, chain parent, expiry, token ceiling, use budget, and audit identity can all be bound without reducing the model's hidden dimension. Exact cumulative use or token limits additionally require the online reservation ledger described above.

Intra-model FFN partitioning ($R_{\text{FFN}} > 1$). When multiple regimes share a single backbone, the gate acts at FFN boundaries, either before the MLP or at its expanded intermediate activation. The following components remain shared across regimes and are *not* covered by the per-regime confinement guarantee:

- **Attention mechanism.** $\mathbf{QK}^\top$ computes a full inner product; the gate cannot act before the entanglement.
- **Token embeddings.** The input embedding table is shared; all regimes map the same vocabulary to the same initial vectors.
- **Output projection.** The language model head (unembedding matrix) is shared.
- **Layer normalization.** LayerNorm statistics are computed over all active dimensions.

For the intermediate placement, the theorem confines the aligned FFN fine-tuning delta when shared components are frozen and the optimizer respects the coordinate partition. The pre-MLP placement has the narrower surface in Table 2. The present classification experiments do not establish content-specific deposition. Shared components remain infrastructure inside the trust boundary.

The honest summary is surface-specific. A whole-model gate provides atomic authorization without changing the authorized computation. A pre-MLP gate zeros selected FFN-input coordinates. The evaluated intermediate gate proves structural coordinate exclusion and aligned FFN-slice confinement, while the shared backbone and runtime-scoped state remain inside the declared trust boundary.

Attention as a duplicated lane. The evaluated construction leaves attention shared and gates only the FFN intermediate activation. In principle, attention could be moved outside the shared boundary only by duplicating complete Transformer lanes, including normalization and every residual route, and gating before recombination. That architecture has substantial compute and operator-error risk and is outside this paper's evaluated scope. No result below relies on multi-regime attention isolation.

Gate placement constraint. A gate's location is part of the security mechanism, not a performance-neutral implementation choice. If protected signal is mixed into active coordinates before masking, the later zero cannot undo that transfer; if a residual, adapter, cache, or other branch bypasses the gate, that branch lies outside the isolation surface. Improper placement can therefore nullify the claimed isolation even though multiplication at the misplaced gate still produces literal zeros.

A concrete negative result sharpens this constraint. Placing LayerNorm *between* the activation $\mathbf{h}$ and the gate $\tilde{\mathbf{h}} = \mathbf{h} \odot \mathbf{g}_r$ breaks $\mathbf{W}_1$ confinement (proven: `non_preserving_breaks_w1_confinement` in Lean 4). LayerNorm normalizes across all dimensions, mixing information from gated-off dimensions back into gated-on dimensions before the gate can act. The safe placement is act $\to$ gate $\to \mathbf{W}_2$, with LayerNorm either before $\mathbf{W}_1$ or after $\mathbf{W}_2$. For the narrower pre-MLP embodiment, the gate must be immediately upstream of $\mathbf{W}_1$ with no intervening mixing operation; it does not inherit the intermediate $\mathbf{W}_1/\mathbf{W}_2$ slice theorem.

6 Formal Verification in Lean 4

The checked-in proof inventory contains 21 Lean modules and 312 active `theorem`/`lemma` declarations. A clean build under the pinned Lean/Mathlib toolchain contains no active `sorry` or `admit` declarations. The core Hadamard-gate, gradient, and aligned FFN weight-confinement theorems are unconditional algebraic results; a checked-in `#print axioms` audit shows that the nine headline algebraic and placement theorems use only Lean/Mathlib's standard logical axioms.

The source tree separately contains five project-specific `axiom` declarations. Two declare opaque predicates and one states a conditional PRF enumeration bound:

- `Recovers` (opaque predicate)
- `prf_brute_force_optimal`
- `IsDistributionMatched` (opaque predicate)

They are not premises of the core gate-zeroing theorems.

Two further statistical axioms are retained in historical conditional modules so that the old theorem chain remains inspectable:

- `camouflage_indistinguishable`
- `gradient_probing_hard`

They are excluded from this paper's claim set and from the headline axiom audit. Their justifications required statistical distribution-matching of injected noise, which is not provable against white-box weight inspection: an adversary holding the model locally can classify dimensions by *function* (coherent activations and gradient signal on regime-appropriate inputs), and marginal distribution matching does not constrain functional distinguishers. Historical custody proposals are not part of the bundled theorem inventory, so this release makes no custody theorem claim. Weight opacity arising from cotraining is retained only as an *observed* hardening property—an empirical claim measured, not a theorem proved—and is a basis for no security statement in this paper.

6.1 Proof Architecture

Selected proof modules are summarized below:

Table 3: Lean 4 proof files and their scope.

File	Scope
`GateSecurity.lean`	Core isolation: forward/gradient isolation, weight confinement, mask orthogonality, wrong-mask-reads-zero
`GatePlacement.lean`	Placement constraints: support preservation, LayerNorm negative result, residual block compositionality
`ModelSecurityV3.lean`	Key hierarchy: confinement composition and recovery-conditioned chain under the PRF axiom; earlier weight- and partition-indistinguishability claims are excluded from this paper's claim set, with their conditional declarations retained for audit (see axiom list)
`CapacitySecurity.lean`	Cross-mask additive cancellation under disjoint support
`BiometricSecurity.lean`	Biometric enrollment certificate and gated voice-separation conditions
`MultiEncoder.lean`	Multi-encoder: federation, Procrustes alignment
`AttentionLeakage.lean`	Negative result: coordinate masks do not isolate a dense shared attention head

6.2 Selected Key Theorems

6.3 Refinement Map and Verification Boundary

The Lean vectors and matrices are linked to the strict DistilBERT runner as follows. With PyTorch's output-by-input linear-weight layout, mathematical $\mathbf{W}_1[:, j]$ corresponds to row j of

Table 4: Selected theorems from the Lean 4 proof suite.

Theorem	Statement
`gradient_isolation`	If $g_{r,j} = 0$, then the gradient w.r.t. h_j is exactly zero, unconditionally.
`weight_update_confined`	Weight updates to $\mathbf{W}_1$ are confined to columns in $\text{supp}(\mathbf{g}_r)$.
`w2_update_confined`	Weight updates to $\mathbf{W}_2$ are confined to rows in $\text{supp}(\mathbf{g}_r)$.
`masks_orthogonal`	For $r \neq s$: $\langle \mathbf{g}_r, \mathbf{g}_s \rangle = 0$.
`access_requires_exact_mask`	If a candidate mask reproduces the gated output for every activation and every $\mathbf{W}_2$, it equals the assigned mask coordinate-wise.
`compose_disjoint`	Union of disjoint masks preserves constituent access.
`residual_block_full_confinement`	FFN residual-branch backward chain confines aligned $\mathbf{W}_1/\mathbf{W}_2$ updates for arbitrary upstream gradient.
`non_preserving_breaks_w1_confinement`	LayerNorm between activation and gate breaks confinement.
`adversary_sees_zero`	For $r \neq s$, applying $\mathbf{g}_s$ gives exactly zero on every coordinate owned by regime r; the active coordinates are those owned by s.
`rejection_sampling_count`	Residue classes in the accepted rejection-sampling range have equal cardinality; PRF pseudorandomness is a separate assumption.

`ffn.lin1.weight`; mathematical $\mathbf{W}_2[j, :]$ corresponds to column j of `ffn.lin2.weight`. The activation $\mathbf{h}$ is the tensor presented to each `ffn.lin2` forward pre-hook, and $\mathbf{g}_r$ is the Boolean 3,072-coordinate mask cast to that tensor's dtype. The research `ScopedAdam` stores first and second moments at the same authorized entries. Execution scope resolution selects the mask before the model call.

Lean verifies the algebraic specification, not the Python hooks, tensor-axis mapping, optimizer code, IdP, or absence of undeclared bypasses. The executable experiments therefore snapshot every frozen and inactive tensor and assert exact zero deltas. A conforming implementation should additionally require deliberately misplaced-gate and bypass controls to fail. The public CPU harness includes both a LayerNorm-before-gate control and an ungated-residual control. These controls exercise the Python placement logic; the Lean LayerNorm counterexample proves the corresponding mathematical failure, but neither is a machine-code verification of a deployment.

The key security assumption underlying the key hierarchy is that HMAC-SHA256 is a pseudorandom function.

7 Support-Constrained Training

An unexpected empirical finding is that the gate's measured performance cost is much smaller than a naive capacity argument would predict. Halving the available dimensions might, in a linear model, appear to halve representational capacity. In five-seed paired runs, co-trained DistilBERT remains within 0.14–0.38 percentage points of separately trained models across R=8–128, including R=96 with only 8 dimensions per regime on a 768-dimensional model. Each independently trained inscription has ordinary optimization variance; the empirical question is the size and uncertainty of the performance cost, not exact equality.

We call this *support-constrained training*. Let $A_r = \text{supp}(\mathbf{g}_r)$. The gated FFN or classifier

contribution at a gated vector is

$$\sum_{j \in A_r} h_j \, \mathbf{W}[j, :],$$

because every term with $j \notin A_r$ is multiplied by zero. Therefore every loss-reducing FFN contribution available to regime r must be represented through its active support A_r. Across jointly optimized regimes, each disjoint support must be sufficient for its assigned task examples. This is a direct linear-algebraic consequence of the gate, not a causal hypothesis about a particular microscopic feature geometry. The empirical question is whether optimization can satisfy the constraint at acceptable task cost; Section 8.2 shows that it does at the formative post-encoder classification surface. Section 8.5 then instantiates the constraint at every intermediate FFN of a frozen shared DistilBERT backbone. Its monotone accuracy decline from R=1 to R=16 directly measures the cost of reducing each tenant from 3,072 to 192 FFN coordinates per layer.

7.1 Gate-Aware Task Training

Two training protocols are evaluated. The formative five-seed models are optimized for the downstream classification task with a mask on the final pooled representation during every forward pass. Because that mask is downstream of the encoder, the protocol does not test intermediate-FFN gradient or parameter confinement. The strict three-seed protocol instead freezes every non-governed shared path, places a mask before each FFN down projection, updates only aligned FFN parameter slices with regime-scoped optimizer state, and selects a private classifier under the same regime identity.

8 Classification-Surface, Cotenancy, and Extraction Validation

8.1 Setup

The formative classification experiment applies a regime mask to the final pooled encoder vector before a shared classifier; it does not place the mask inside each Transformer FFN. We report them as formative evidence that gate-aware optimization can learn useful support-constrained representations, not as empirical FFN-placement evidence. DistilBERT is evaluated on AG News [Zhang et al., 2015] four-class classification (8,000–20,000 training examples and 7,600 test examples). We then separately test standalone extraction of gated projection columns. Paired statistical tests use five seeds where explicitly reported. The strict cotenancy experiments below instead gate every expanded intermediate FFN and use three seeds unless stated otherwise.

Table 5: Post-encoder classification configurations.

Model	Hidden dim	Params	R tested	Training
DistilBERT-base	768	66M	8–128	Co-trained

8.2 Co-Training Cost Versus Separate Controls

The updated DistilBERT sweep uses five paired seeds at each of seven partition ratios on parallel A100 workers. In this formative protocol, the mask acts on the 768-dimensional final CLS vector; "Dims/Regime" below therefore refers to pooled-classifier input coordinates, not DistilBERT's 3,072-dimensional intermediate FFN. At every ratio, the gated model is trained across all R regimes, whereas the separate-model control samples four regimes; each per-seed statistic pairs the mean of those four controls with the corresponding four gated-regime accuracies.

Across the seven ratios, the measured cost is small: the separate-minus-gated gap ranges from 0.143 to 0.376 percentage points. The R=96 paired test is significant at the unadjusted

Table 6: Five-seed paired co-training results (DistilBERT, AG News). Gap is separate minus gated accuracy in percentage points. Intervals use Student's t distribution with four degrees of freedom.

R	Dims/Regime	Gap (pp)	95% CI (pp)	p	Bonf. reject?
8	96	0.348	[−0.229, 0.925]	0.1692	No
16	48	0.268	[−0.301, 0.837]	0.2616	No
24	32	0.208	[−0.324, 0.740]	0.3392	No
32	24	0.267	[−0.345, 0.879]	0.2920	No
64	12	0.143	[−0.041, 0.326]	0.0967	No
96	8	0.376	[0.045, 0.708]	0.0345	No
128	6	0.272	[−0.208, 0.752]	0.1902	No

0.05 level, but no test crosses the Bonferroni family-wise threshold $0.05/7 \approx 0.0071$. Bonferroni is intentionally conservative; we include it because seven related hypotheses were inspected. The ratios share the same paired seed schedule and are correlated, and the R=96 result is treated as an exploratory post-hoc observation rather than a preregistered discovery. Failure to reject a zero difference is not proof of equality, nor is exact equality expected from independently optimized inscriptions. The defensible conclusion is that the observed performance costs are small in this five-seed DistilBERT study. A formal equivalence claim would additionally require a predeclared practical margin and an interval wholly contained within that margin.

The archived experiment initially printed normal-approximation intervals using 1.96 despite only five pairs. Table 6 recomputes the intervals with the matching Student-t critical value; this correction also removes an apparent inconsistency between the R=64 interval and its paired-test p-value.

Replay. Use the legacy post-encoder surface in `experiments/reproduce_distilbert_classification.py`; the reported five-seed rows are retained in `experiments/results/series1_results_20260803_202919.json`.

8.3 Holding Per-Regime Classifier Capacity Fixed

The preceding sweep fixes the 768-dimensional post-encoder representation, so each support narrows as R grows. This is not the only sizing policy. If a task needs q hidden coordinates per regime, the governed layer may instead choose $d = qR$. Adding a regime then adds a block without reducing any incumbent regime's allocated width.

We ported and reran the formative wide-classifier protocol with R=128, q=10, and therefore d=1,280. The two-layer MLP has 256 one-hot synthetic observations and 256 disjoint answers, two per regime. It is co-trained for 2,000 full-batch epochs using a post-activation binary gate. The sparse selected-support implementation is also compared with explicit dense-mask execution.

The owning key produces the expected answer for 256/256 observations. Across all 32,512 pairs of a foreign observation and wrong regime key, the expected foreign answer is never produced. Activating all 128 regimes simultaneously produces 249/256 correct answers, and selected-support versus dense-mask logits have maximum absolute difference zero.

This result is deliberately narrow. It is a deterministic synthetic memorization and scalability test with no held-out set, and the zero wrong-key hit count is an empirical negative control rather than a privacy theorem. It does show why "R regimes" need not imply "$1/R$ of a fixed model" for modular classifiers: each regime retains the same ten-coordinate allocation because the governed layer grows with R. The cost is linear parameter storage, not lost per-regime width.

Replay. The self-contained runner is `experiments/capacity_preserving_wide_classifier.py`; the reported run is `experiments/results/capacity_preserving_wide_20260820_225331.json`.

8.4 Deployment Service-Consolidation Consequence

An R=8 DistilBERT SST-2 deployment benchmark compares eight complete model services with three single-backbone layouts. Eight services require 1.071 GB of checkpoint tensors and 1.103 GB of resident CUDA parameter and buffer state. A frozen shared backbone with eight zero-initialized private adapter slots retains the original 91.06% accuracy using 151.6 MB and 156.2 MB, respectively. This is a deployment control, not evidence that untrained adapters add utility.

Post-hoc one-eighth FFN slicing reduces accuracy to 67.66%. Physically extracting the authorized slices preserves the sliced logits within the declared fp16 tolerance and raises measured throughput from 1,585 to 2,872 samples/s. Every scored request passes through execution authority and malformed authority makes zero model calls. Thus the measurement supports consolidating a frozen backbone plus private modules, while directly rejecting the stronger claim that naive narrowing of a trained dense Transformer preserves utility. Timing is one serial stream on one NVIDIA T4 and is not a concurrency result.

Replay. Run `experiments/modal_distilbert_service_consolidation.py`; the full artifact is listed below.

`experiments/results/distilbert_service_consolidation_20260821T135530_707107Z.json`.

8.5 Strict Frozen-Backbone Transformer Cotenancy

The formative post-encoder study above establishes task feasibility, not a separated shared-model state boundary: its optimizer may update attention, normalization, and other shared encoder parameters. We therefore run a second protocol whose acceptance condition is stricter than task accuracy. DistilBERT's attention, embeddings, normalization, residual parameters, and all other non-governed shared weights are frozen. A gate is placed on the 3,072-dimensional expanded activation immediately before every FFN down projection. For regime r, training may update only the active rows of $\mathbf{W}_1$, the matching coordinates of $\mathbf{b}_1$, the matching columns of $\mathbf{W}_2$, and a private regime-selected classifier in PyTorch's `Linear.weight` storage convention. Those tensors are transposes of the row-vector notation in (6), where the same slices are columns of $\mathbf{W}_1$ and rows of $\mathbf{W}_2$. A regime-scoped Adam implementation updates moments only on those authorized entries; ordinary Adam momentum would otherwise continue moving a previously active coordinate during a later tenant's step even when its current gradient is zero.

We train on 8,000 AG News examples for two epochs and evaluate on 2,000 held-out examples at seeds 42, 123, and 256. Label permutations provide regime-specific functional identities. The R=1 condition is the full-FFN-capacity reference under the same frozen-backbone protocol.

Table 7: Strict intermediate-FFN cotenancy on frozen DistilBERT. Accuracy is mean ± sample standard deviation across three seeds. The drop is relative to the matched R=1 capacity reference.

R	FFN dims/tenant/layer	Owning accuracy	Drop (pp)	Max unauthorized delta
1	3,072	$91.28\% \pm 0.33\%$	0.00	0
2	1,536	$90.15\% \pm 0.39\%$	1.13	0
4	768	$88.93\% \pm 0.30\%$	2.36	0
8	384	$87.47\% \pm 0.04\%$	3.82	0
16	192	$86.21\% \pm 0.08\%$	5.07	0

"Max unauthorized delta" is the maximum over frozen shared parameters, off-partition FFN parameters, off-partition first and second optimizer moments, and inactive private classifiers in all runs. Every category is bit-exact zero. Thus the capacity cost is empirical and monotone in this setup, while the declared state boundary is exact. Halving each FFN retains 90.15% accuracy, 1.13 percentage points below the full-capacity reference. At R=16, the 192-coordinate slice incurs

a material but non-catastrophic 5.07-point drop. This is a task- and architecture-specific capacity curve, not a universal Transformer scaling law.
Replay. Run `experiments/modal_dense_ffn_cotenancy.py`; the three-seed artifacts are the `experiments/results/dense_ffn_cotenancy_*.json` files and their aggregate is `experiments/results/transformer_cotenancy_summary.json`.

Preliminary public gate-aware adaptation. The preceding sweep applies intermediate-FFN masks post hoc to the pretrained backbone. We separately test whether a backbone can learn the R=8 partition geometry without receiving tenant data. At each of seeds 42, 123, and 256, an ungated teacher is fine-tuned for two epochs on 8,000 examples from a designated public AG News split. A student copied from that teacher is then trained for one epoch on the same public split; for every batch, it is evaluated through each of the eight 384-coordinate FFN masks and optimized with an equal mixture of hard-label loss and temperature-2 teacher distillation loss. The comparison checkpoint retains the teacher without this additional epoch and is therefore a no-extra-training baseline, not an equal-compute control. The next 8,000 shuffled training examples form a disjoint tenant-stage split. For both conditions, all shared paths are then frozen and only aligned FFN slices and private classifiers are trained for two epochs, exactly as in the strict protocol above. The first 2,000 standard test examples are held out from both stages.

Table 8: Preliminary R=8 public-adaptation comparison. Accuracy is mean ± sample standard deviation across three seeds. Uplift is relative to the no-extra-training baseline.

Initialization	Owning accuracy	Difference (pp)	Max delta
Public task only	$88.55\% \pm 0.11\%$	0.00	0
+ all-mask distillation	$90.15\% \pm 0.11\%$	1.60 ± 0.10	0

Before tenant training, the ungated teacher reaches $91.62\% \pm 0.35\%$. Applying the eight masks to the generic-public control yields $74.60\% \pm 5.42\%$ mean accuracy, while the all-mask student reaches $89.89\% \pm 0.51\%$. After the strict tenant stage, the combined extra-training, masking, and distillation pipeline retains the 1.60-point difference and finishes 1.47 points below the ungated teacher. Frozen shared parameters, off-partition FFN parameters and optimizer moments, and inactive classifiers all remain bit-exact zero during tenant training. Because the baseline did not receive the extra public epoch, this comparison cannot identify whether the difference comes from additional training, mask-aware adaptation, distillation, or their interaction. It is retained as preliminary pipeline evidence, not a causal estimate and not evidence of zero utility loss. The "public" and "tenant" designations are a controlled disjoint split of one public benchmark, and label permutations supply functional identities; real private-corpus generalization remains outside this test.
Replay preliminary adaptation. The reported three-seed artifact is `experiments/results/public_gate_distillation_20260806T193105_795325Z.json`. The repository retains the stronger equal-work successor below, but not the exact historical runner that produced this artifact.

Equal-work factorial result. The checked-in successor runner copies the same public teacher into four conditions: no extra public adaptation, an extra ungated hard-label epoch, an extra all-mask hard-label epoch, and an extra all-mask hard-label-plus-distillation epoch. The three adapted conditions use the same examples, batch order, optimizer steps, teacher evaluations, and student forward/backward counts; the ungated arm repeats each batch R times to match model-pass counts. The resulting paired contrasts estimate the effects of extra training, mask-aware training, and distillation conditional on masks. In the completed full seed, ordinary extra training changes tenant accuracy by +0.075 percentage points, mask awareness adds +1.331 points beyond that control, and distillation adds +0.031 points conditional on masks. The

full pipeline is +1.438 points above no extra adaptation, with exact zero frozen, off-partition, optimizer-moment, and inactive-classifier deltas. The reduced pilot is negative and retained as a protocol check. One full seed does not support a population-level effect claim.
Replay factorial. Run `experiments/modal_public_gate_adaptation_factorial.py`; the completed artifact is `experiments/results/public_gate_adaptation_factorial_20260821T063703_914999Z.json`.

Complete private lanes on shared attention. We also test two designs that avoid dividing a trainable tenant module by R. Both keep the pretrained DistilBERT backbone frozen and shared. The *adapter* design adds a 64-dimensional private residual adapter after every Transformer block; the *expert* design adds a full 3,072-dimensional private FFN after every block. The regime-selected classifier is private in both designs. The private output then enters the next frozen shared attention block: attention is useful and unablated, but remains inside the trusted shared boundary.

Table 9: Complete private lanes on a frozen shared DistilBERT backbone (R=4). Accuracy is mean ± sample standard deviation across three seeds.

Design	Owning accuracy	Wrong-key accuracy	Max shared/inactive delta
64-dim residual adapters	$90.47\% \pm 0.18\%$	$3.18\% \pm 0.06\%$	0
3,072-dim private experts	$90.09\% \pm 0.14\%$	$3.30\% \pm 0.05\%$	0

The private adapter is the best measured tradeoff here: it slightly outperforms the much larger expert while adding approximately 0.59M trainable parameters per tenant rather than 28.33M. The wrong-key values are functional negative controls induced by distinct label permutations; they are not the source of the security claim. The exact frozen-backbone and inactive-lane deltas establish that tenant training state does not enter another lane under the declared optimizer and runtime boundary. Shared activations are not claimed to be tenant-private, and cache, batching, and serving-runtime state must be separately namespaced.
Replay. Run `experiments/modal_private_transformer_lanes.py`; the reported adapter and expert runs are `experiments/results/private_transformer_lanes_*.json`.

Key-authorized customer corpora and Cargo Mode. Finally, a legacy two-tenant smoke test connects the model-state boundary to runtime data authorization. Each customer corpus occupies a separate vector-store partition. The lockbox side retains the authorization root key and grants a scoped Cargo access key. The completed artifact binds tenant and regime but predates the canonical sub-id requirement. It is retained as a real-model mechanism smoke, not as evidence that the current complete scope was enforced. Its manifest also binds the embedding specification, payload hash, and item count; payload count and hash are validated before ingest. Retrieval must successfully dock at the corresponding regime bus before context is passed to an otherwise normal shared FLAN-T5 generator.

All four owning questions retrieve the correct customer partition and produce the exact canary answer. Six unauthorized attempts (random key, other-tenant key, and other-regime key against each tenant) are rejected before retrieval, with zero unauthorized model calls. All load and retrieval receipts verify under the scoped access keys. For this retained legacy artifact, "verify" denotes the recorded signature check; it is not evidence that the receipt committed to the exact returned content. These small counts are protocol checks, not statistical leakage estimates. The enforcement claim depends on the vector store, master key, lockbox proxy, and runtime process remaining trusted and on corpus access being exposed only through the regime bus. This design lets ordinary Transformer attention consume authorized customer context; it does not partition or ablate attention heads.

The checked-in successor protocol is self-contained and enforces the complete research scope. It binds tenant, sub-id, regime, model digest, operation, policy version, and the registered vector partition, with distinct load and retrieval keys plus wrong-sub-id, wrong-operation, and same-regime wrong-partition negative controls. It now also places a separate tenant + sub-id + model + operation gate before generator invocation. It is configured as a protocol rerun; no result from that strengthened GPU runner is claimed until its artifact is checked in. The completed local CPU protocol check exercises the same fail-closed scope logic, including wrong-sub-id and whole-model negatives with zero unauthorized model calls, without making a generative-quality claim.

Replay. Run `experiments/local_transformer_cotenancy_suite.py` for the Gate-backed local scope matrix or `experiments/modal_cargo_transformer_authorization.py` for the real-model Cargo experiment. Retained outputs and their evidentiary status are indexed by `experiments/results/README.md`.

8.6 Structural FFN Cross-Regime Coordinate Exclusion

For $r \neq s$, disjointness gives $\mathbf{g}_r \odot \mathbf{g}_s = \mathbf{0}$: a runtime invocation resolved to regime s cannot activate coordinates owned by regime r. The lockbox generates the global Fisher–Yates assignment and scoped capabilities; the trusted runtime performs credential-to-mask dispatch. Callers do not choose a permutation directly. This coordinate exclusion is the structural guarantee. The formative runner also computes wrong-key task accuracy, but the checked-in parity-only artifact has an empty scaling block and therefore does not contain the per-regime or cross-regime measurements previously quoted here. We make no numerical wrong-key claim from that formative artifact. The distinct strict-cotenancy runs in Section 8.5 do archive wrong-key functional controls, but their security statement rests on exact inactive state and runtime rejection rather than low task accuracy. The exact coordinate zero does not depend on either task-level measurement.

8.7 Deployment Efficiency

A shared gated deployment stores one backbone plus masks and any private heads or lanes, whereas a baseline that materializes R identical architecture copies stores the backbone R times. At R=4, the checked-in CPU smoke reproduces the resulting 4× raw state-dict arithmetic for the identical-copy baseline; it does not establish utility equivalence or complete production memory savings. A public runner also measures simultaneous accelerator allocation and routed versus multiplexed throughput, but the repository does not yet contain a full GPU artifact supporting the historical 3.6× memory and 2.69× throughput figures. Those figures are withheld pending a hardware-matched rerun. The benchmark applies per-sample masks at the pooled classifier input and is not an intermediate-FFN systems benchmark.

Replay measurements. Storage, live accelerator allocation, and routed-throughput collection are implemented in `experiments/reproduce_distilbert_deployment.py`; the retained publication-safe artifact is `experiments/results/distilbert_deployment_20260806_103232.json`.

8.8 Exact Submodel Extraction

This experiment asks whether an authorized regime can be exported as a smaller standalone linear projection without changing the function computed by the full gated projection. For a full matrix $\mathbf{W} \in \mathbb{R}^{4\times 768}$, input batch $\mathbf{X} \in \mathbb{R}^{256\times 768}$, bias $\mathbf{b}$, and active coordinate set A_r, we compare

$$(\mathbf{X} \odot \mathbf{g}_r)\mathbf{W}^\top + \mathbf{b} \quad \text{with} \quad \mathbf{X}[:, A_r]\,\mathbf{W}[:, A_r]^\top + \mathbf{b}. \tag{20}$$

The left side executes the full matrix after applying the gate; the right side physically removes every inactive input coordinate and executes only the extracted columns. Equality is expected

algebraically because all omitted products are multiplication by zero. The empirical test determines whether implementation details or floating-point reduction order create a numerical difference.

Strategy. For each of 10 random seeds, we generate a random input batch, weight matrix, bias, and permutation of 768 coordinates. The permutation is split into four disjoint 192-coordinate regimes. We test each regime individually and then test authorized unions containing one, two, three, and all four regimes. The complete matrix is repeated in 32-bit and 16-bit floating point, yielding 80 single-regime checks and 80 compound-mask checks. The predeclared absolute tolerances are 10^{-6} for 32-bit and 2×10^{-2} for 16-bit, with an additional 16-bit relative tolerance of 10^{-3}.

Table 10: Exact-extraction matrix. "Bit-equal" counts comparisons whose output tensors have identical bits; all 160 comparisons satisfy the declared numeric tolerance.

Precision	Comparisons	Bit-equal	Within tolerance	Max $\lvert\Delta\rvert$
32-bit floating point	80	80	80	0
16-bit floating point	80	60	80	0.015625

The result confirms exact execution in 32-bit arithmetic for this matrix and tolerance-bounded equivalence in 16-bit arithmetic. It matches the Lean 4 gated-compressed equivalence theorem (`compressed_eq_gated`), while making the finite-precision qualification explicit. For one of R equal-width regimes, the extracted projection retains $1/R$ of the gated input-facing weight columns; authorized compound masks retain the union of their columns. This is also the linear algebra of extracting the active input rows of $\mathbf{W}_1$ in the row-vector pre-MLP convention. The separate local FFN protocol check instantiates a $32 \to 64$ first projection and observes zero inactive input and $\mathbf{W}_1$-row gradients. Neither check is a pre-MLP task-utility benchmark.
Replay. Run `experiments/local_exact_extraction.py`; the reported artifact is `experiments/results/local_exact_extraction_20260803_172654.json`.

8.9 Excluded Experimental Scope

The empirical claims in this section include post-encoder classification, strict intermediate-FFN and private-lane cotenancy, the keyed Cargo retrieval smoke, checked-in storage, exact extraction, and local pre-MLP/whole-model authorization protocol checks. Exploratory post-pooled embedding, open-ended generative tenant-state training, complete attention-lane partitioning, adapter-state exfiltration, and KV-cache studies remain excluded: they either do not instantiate the declared intermediate FFN/runtime boundary or address systems beyond this paper's scope. Their artifacts remain archived for future work but are not used as evidence here.

9 Supplementary Keyed Permutation Conjugation

The previous sections partition a model's hidden dimensions into disjoint subspaces using binary gate masks. A complementary question arises: can a single trained function be assigned R keyed basis placements by exploiting the fact that a transformer's computation is equivariant under permutation of the residual-stream basis? The permutation creates the keyed placement, not tenant knowledge or an additional isolation boundary.

9.1 Permutation Equivariance of Transformers

Theorem 3 (Residual-stream permutation equivariance)**.** *Let $\mathcal{T}$ be a transformer with residual-stream dimension d, and let $P \in \{0,1\}^{d \times d}$ be a permutation matrix. Write residual activations as*

row vectors and relabel them by $\mathbf{x} \mapsto \mathbf{x}P^\top$. *Define the* conjugated *transformer* $\mathcal{T}^{(P)}$ *by replacing every residual-facing parameter:*

$$\mathbf{W}_{\text{emb}} \mapsto \mathbf{W}_{\text{emb}}P^\top \qquad \textit{(embedding outputs)} \tag{21}$$

$$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \mapsto P\mathbf{W}_Q,\ P\mathbf{W}_K,\ P\mathbf{W}_V \qquad \textit{(attention inputs)} \tag{22}$$

$$\mathbf{W}_O \mapsto \mathbf{W}_O P^\top \qquad \textit{(attention output)} \tag{23}$$

$$\mathbf{W}_1^{\text{FFN}} \mapsto P\mathbf{W}_1^{\text{FFN}} \qquad \textit{(FFN input)} \tag{24}$$

$$\mathbf{W}_2^{\text{FFN}} \mapsto \mathbf{W}_2^{\text{FFN}}P^\top \qquad \textit{(FFN output)} \tag{25}$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta} \mapsto \boldsymbol{\gamma}P^\top,\ \boldsymbol{\beta}P^\top \qquad \textit{(LayerNorm)} \tag{26}$$

$$\mathbf{b}_O, \mathbf{b}_2 \mapsto \mathbf{b}_O P^\top,\ \mathbf{b}_2 P^\top \qquad \textit{(residual-output biases)} \tag{27}$$

$$\mathbf{W}_{\text{cls}} \mapsto P\mathbf{W}_{\text{cls}} \qquad \textit{(classifier input)} \tag{28}$$

Then $\mathcal{T}^{(P)}$ *produces identical outputs to* $\mathcal{T}$ *on all inputs:*

$$\mathcal{T}^{(P)}(\mathbf{x}) = \mathcal{T}(\mathbf{x}) \quad \forall\, \mathbf{x} \tag{29}$$

Proof sketch. The conjugation relabels the residual-stream coordinate system. At every input-side map, $\mathbf{x}P^\top(P\mathbf{W}) = \mathbf{x}\mathbf{W}$; every residual-output map appends $P^\top$. LayerNorm's mean and variance are permutation-invariant, and its scale and shift are relabeled by the same $P^\top$, so LayerNorm commutes with the conjugation. Attention computes $\mathbf{Q}\mathbf{K}^\top$ on the *per-head* subspace, which is an internal coordinate system unaffected by the residual-stream permutation once $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$ absorb P on their input side. The result is a change of basis that is invisible to the model's computation. □

The theorem implies that for any trained model and any $R \leq d!$ distinct permutations $\{P_0, P_1, \ldots, P_{R-1}\}$, the R conjugated models $\{\mathcal{T}^{(P_r)}\}$ are functionally identical yet operate in mutually distinct coordinate systems. Each conjugated model is a keyed *basis placement*: the accuracy gap from the base model is exactly zero, not approximately zero. Conjugation alone does not create distinct knowledge or zero non-owning coordinates. A disjoint CDP mask can provide the coordinate zeros in the selected basis; tenant-specific deposition into those coordinates is not evaluated here.

Relationship to CDP. Permutation conjugation and CDP's Hadamard gate are complementary mechanisms. CDP partitions dimensions via binary masks to provide FFN-coordinate confinement. Permutation conjugation relabels the entire coordinate system to achieve *keyed basis placement* (different regimes use different coordinate systems for the same initial computation). The two compose naturally for a single selected basis: an outer permutation conjugation places the model in that orientation, and an inner CDP gate selects FFN coordinates within it. This composition preserves the gate theorem but does not itself establish tenant-specific learning. The credential, custody, and dispatch assumptions required for this composition are stated in Section 4.

Operationally, the inner gate and outer shuffle perform different jobs. The binary mask first determines which FFN coordinates are active; the residual-stream permutation then removes a stable public labeling of the surrounding representation and binds its realization to the engine-held key. For an FFN gate, P_r is absorbed at the residual-facing sides of $\mathbf{W}_1$ and $\mathbf{W}_2$, while $\mathbf{g}_r$ acts on the internal FFN activation between them. Thus the two operations do not compete: ablation determines the active FFN support, and the keyed shuffle determines the coordinate system in which the trusted runtime realizes it. If a gate and permutation act on the same coordinate space, the equivalent regime-local mask is simply $P_r\mathbf{g}_r$.

The reference code contains the conjugation, multiplexing, paper-aligned intermediate FFN gate, and nested-mask paths. The algebraic composition is supported by `permute_hadamard` and

`permutation_preserves_logits` in Lean 4. These paths are not yet packaged as one unified production execution mode; the security claim for a deployment follows the specific composed path it actually instantiates.

9.2 Empirical Validation

We validate residual-stream permutation conjugation on DistilBERT-base [Sanh et al., 2019] (66M parameters, d=768, 6 layers, 12 heads) fine-tuned on AG News [Zhang et al., 2015] 4-class classification (8,000 train / 7,600 test, 4 epochs, AdamW lr=2×10^{-5}, seed 0). All experiments run on Apple MPS (M-series laptop). Base accuracy: 90.25%.

Experiment A: R-sweep. Train one model. For each $R \in \{4, 8, 16, 32, 64, 128\}$, generate R random permutations (seeded), deep-copy the model, conjugate all residual-facing parameters per Theorem 3, and evaluate on the full test set. At R=1024, sample 8 regimes across the full range ($r \in \{0, 1, 10, 100, 500, 777, 999, 1023\}$).

Table 11: Keyed permutation-conjugation R-sweep. Accuracy gap is exactly zero at every R tested, from 4 to 1,024.

R	Regimes tested	Max $\lvert\Delta\text{acc}\rvert$	Accuracy
4	4	0.00	90.25%
8	8	0.00	90.25%
16	16	0.00	90.25%
32	32	0.00	90.25%
64	64	0.00	90.25%
128	128	0.00	90.25%
1,024	8 (sampled)	0.00	90.25%

The measured accuracy gap is zero at every regime tested, including regime indices 777 and 1023 at R=1024. This prediction-level result is consistent with the theorem; exact functional identity follows from the conjugation algebra rather than from finite test-set accuracy alone.

The experiment tests an expected invariance of a single function; it does not establish additional learned functions, tenant state, or an isolation boundary. A permutation vector is only addressing metadata for that reparameterization, so comparison with materialized copies of the same function is not presented as model compression.

Experiment B: component ablation. To verify that conjugation must be total, we systematically skip conjugation for each component class (embeddings, LayerNorm, attention projections, FFN weights, classifier) and measure the resulting accuracy change.

In this implementation, omitting every tested component changes accuracy. The most destructive single omission in this run is the V-projection (collapse to 21.86%, below the 25% chance baseline), but the ablation does not establish a general ranking of information paths. Even the embedding LayerNorm causes a 1.66 pp gap when skipped, because its element-wise scale and shift parameters are residual-facing.

9.3 Keyed-Basis Batched Inference

Permutation conjugation can batch R keyed basis placements through one set of weights. This is a systems implementation of function-preserving reparameterization, not tenant isolation or a new batching primitive. Instead of pre-conjugating model copies, we apply per-sample permutations to the activations at every residual-stream boundary via `torch.gather`:

1. Before each weight matrix (input side): inverse-permute the activations to the original basis so the shared weights apply correctly.

Table 12: Component ablation: skipping each component from conjugation changes accuracy, often severely. Base accuracy 90.25%.

Skipped component	**Accuracy**	**Gap**
None (full conjugation)	90.25%	0.00
Embeddings	24.72%	−65.53
Embedding LayerNorm	88.59%	−1.66
SA LayerNorm	39.89%	−50.36
Output LayerNorm	43.41%	−46.84
All LayerNorm	41.11%	−49.14
Q projection	81.95%	−8.30
K projection	77.07%	−13.18
V projection	21.86%	−68.39
All Q/K/V	21.68%	−68.57
Out projection ($\mathbf{W}_O$)	24.96%	−65.29
FFN $\mathbf{W}_1$	25.00%	−65.25
FFN $\mathbf{W}_2$	52.21%	−38.04
FFN (both)	27.80%	−62.45
Classifier	45.78%	−44.47

2. After each weight matrix (output side): forward-permute the result back to the sample's regime basis.
3. Around each LayerNorm: inverse-permute, normalize, then forward-permute (because LayerNorm parameters are element-wise in the original basis).

The weight matrices are never modified; only the activations are shuffled per-sample. Correctness was verified at R=4: the maximum logit difference between keyed-basis batched and serial (pre-conjugated copy) evaluation is 1.19×10^{-6}, with bit-exact predictions.

Table 13: Keyed-basis batching: time to evaluate R samples (one per basis placement) through a single DistilBERT backbone. Apple MPS, 30 trials, 5 warmup.

R	Keyed batch	Serial	Speedup	Per-sample	Cost vs. R=1
1	2.79 ms	2.66 ms	0.95×	2.79 ms	1.00×
4	6.28 ms	10.73 ms	1.71×	1.57 ms	2.25×
8	10.92 ms	39.24 ms	3.59×	1.37 ms	3.91×
16	21.92 ms	45.53 ms	2.08×	1.37 ms	7.86×
32	43.35 ms	94.60 ms	2.18×	1.35 ms	15.54×
64	90.66 ms	205.44 ms	2.27×	1.42 ms	32.49×
128	339.03 ms	469.94 ms	1.39×	2.65 ms	121.52×

The per-sample cost drops from 2.79 ms (R=1, no permutation) to a floor of 1.35 ms (R=32) as GPU utilization improves with larger batch sizes. At R=128, the per-sample cost rises to 2.65 ms as memory bandwidth saturates on the test hardware. The total cost of evaluating 128 keyed basis placements (339 ms) is 121.5× the cost of one. Ordinary batching improves accelerator utilization while the permutation gathers add per-sample work. The benchmark does not include an unpermuted batch at the same sizes, so it does not isolate permutation overhead from ordinary batching efficiency. It is a correctness and placement benchmark, not evidence of tenant-specific deposition, isolation, or a new batching primitive.

Idealized operation-count ratio. The gather operations contribute $\mathcal{O}(B \cdot \text{seq} \cdot d)$ memory bandwidth per layer. The matrix multiplications contribute $\mathcal{O}(B \cdot \text{seq} \cdot d^2)$. The ratio is

$\mathcal{O}(1/d) \approx 1/768 \approx 0.13\%$. This ratio omits kernel launch, memory movement, indexing, and hardware effects and is not a measured latency floor. The measured overhead (93.8% at R=128 on MPS) is dominated by Python-level `torch.gather` calls at each injection point. A hypothetical fused kernel that permutes weight-matrix indices during the GEMM—or precomputes permuted weight views ($\mathbf{W}[:, P]$)—could reduce this overhead, but no such kernel is implemented or benchmarked here.
Replay. Run `experiments/orthogonal_superposition_sweep.py` for permutation conjugation and `experiments/true_multiplexing_test.py` for keyed basis batching. Retained sweep, ablation, and timing outputs are indexed by `experiments/results/README.md`.

10 Discussion

CDP as measurement instrument. End-to-end cross-tenant leakage is ordinarily estimated statistically—for example with distribution comparisons, membership-inference attacks, or accuracy deltas. CDP adds a local structural zero at its declared FFN gate. The provable baseline is local and exact: an inactive coordinate at the declared gate is zero, so no signal can traverse that coordinate ($0 \cdot x = 0$). The reported 0.499 AUC membership-inference score and 0/30 factual exfiltration result belong to prior work and are not evidence in this paper. Previously quoted wrong-key classification values are also withheld because their scaling artifact is not checked in. A nonzero signal at an excluded coordinate contradicts the construction; a nonzero end-to-end attack metric can also reveal an implementation error or a path outside the declared isolation surface.

Prior work [McCormick, 2026] reports end-to-end leakage measurements using a gated control. Those external attack studies are not reproduced here and are not used to enlarge the local FFN theorem. This paper's empirical evidence is limited to Section 8 and Section 9.2.

Governance relevance. The runtime binds an authenticated grant, model version, derived mask, and audit record before applying the gate. This can support provenance questions such as which FFN coordinates were authorized for an invocation and whether a confined slice was revoked. It does not by itself satisfy a regulatory framework, certify shared model components, or prove deletion of state outside the declared parameter and runtime surface. Nor does deleting a recipient from a new lockbox revoke copied key bytes unless the runtime rejects the old grant ID/key epoch. Under that mandatory online check, principal-scoped derivation permits one tenant + sub-id grant to be revoked without rotating unrelated principals.

Comparison with LoRA isolation. LoRA does not itself provide a cryptographic authorization boundary. CDP structurally eliminates any component of a signal path that must traverse a non-owning gated FFN coordinate; declared shared components remain outside that statement. The relevant distinction is architectural: LoRA adds trainable low-rank updates but does not impose an authorization gate, whereas CDP's local statement follows from Hadamard zeroing at the declared FFN surface.

Limitations. For intra-model FFN partitioning, the primary limitation is the shared backbone trust boundary (Section 5). Attention, embeddings, and the output projection are shared. We do not evaluate leakage through those shared activation paths. The proofs and exact-delta experiments cover the declared FFN/private state boundary, not partitioning of the complete model. Whole-model gating is a separate operational authorization boundary and does not strengthen the local FFN theorem.

The guarantee also depends on gate placement and path closure. A post-mixing gate or an ungated bypass may preserve the local identity $0 \cdot x = 0$ while invalidating the intended end-to-end isolation surface. Implementations must place the FFN gate on the expanded activation before

the down projection for the intermediate theorem, or before the first projection for the narrower pre-MLP surface. Execution-scoped state must follow the same regime boundary wherever it is claimed as confined. Attention-coordinate partitioning and generative tenant-state training remain outside the evaluated scope. Cargo's frozen-generator smoke tests runtime release of retrieved context, not deposition into generative model weights.

The formative five-seed classification sweep gates the final pooled representation rather than the expanded intermediate FFN activation. It therefore demonstrates support-constrained task feasibility at that downstream surface. The separate strict-cotenancy study does place the gate at every expanded intermediate FFN and reports utility and exact parameter/optimizer confinement through R=16. Neither study partitions shared attention coordinates or establishes multi-regime isolation across the internals of a complete model.

The strict experiments use one AG News distribution and label permutations as regime-specific functional identities. This is sufficient to exercise routing, aligned updates, and exact inactive state checks, but it is weak evidence for heterogeneous customer corpora or naturally different tenant tasks. A stronger follow-up should use distinct corpora or task distributions with shared label semantics, paired seeds, and a deliberately broken gate or bypass control.

Broader impact. CDP can support consent-based policy for the declared gated FFN surface. Coordinate exclusion is the default state there; intentional unions require an authorized multi-regime grant. Shared components remain inside the stated trust boundary. The mechanism may be useful where operators need an auditable local state boundary, but it does not itself establish regulatory compliance or an end-to-end privacy guarantee.

Algebraic annihilation (Section 3.7) provides verifiable FFN-slice erasure: the selected $\mathbf{W}_1$, $\mathbf{b}_1$, and $\mathbf{W}_2$ coordinates are zeroed, and the confinement theorem states that regime-scoped updates did not leave that declared parameter surface. Calling this a complete right-to-deletion mechanism additionally requires $\mathbf{b}_2$, optimizer state, adapters, shared components, logs, caches, and backups to be frozen, separately scoped, or erased under the same lifecycle policy.

11 Conclusion

Cryptographic Dimension Partitioning defines two explicit scopes. Within an FFN, a globally assigned binary mask provides exact coordinate zeros at either a pre-MLP input or an expanded intermediate activation; the strongest aligned parameter-state result uses the intermediate placement. Around a complete model, an IdP-bound runtime gate authorizes or rejects the unchanged model as one atomic resource. The core Lean 4 theorems contain no `sorry` declarations or project-specific axioms. Their scope is precise: exact forward and loss-gradient zeros at a correctly placed FFN gate, and aligned parameter-state confinement when the optimizer obeys the same support.

The strict DistilBERT study measures the resulting tradeoff. Across three seeds, owning accuracy falls from 91.28% at R=1 to 86.21% at R=16, while frozen-shared, off-partition parameter, off-partition optimizer-moment, and inactive-classifier deltas remain exactly zero. Private residual adapters and private experts recover 90.47% and 90.09% mean accuracy without modifying the shared backbone or inactive lanes. The separate post-encoder study shows small support-constrained classification costs, but does not validate intermediate-FFN placement. The equal-work public-adaptation factorial separates ordinary extra training, mask-aware adaptation, and distillation conditional on masks; its positive result is one full seed and therefore not a population-level effect estimate.

The authority-constrained MoE study adds a distinct result: learned semantic routing can operate inside a fixed grant-selected expert set. At R=8, packing separately trained routers and experts changes zero held-out logits or predictions while unauthorized dispatch and the audited

inactive training state remain exactly zero. This is lossless module packing, not compression of expert bytes or partitioning of shared attention.

The capability-prefix extension sharpens the control/data boundary at token granularity. Execution authority, not prompt text, selects the private continuous prefix and allowed experts; the learned token router operates only afterward. The held-out spoof control and 311,477 routed-token trace support that bounded claim, but the classifier does not establish autoregressive generation quality.

An intra-model FFN gate does not partition attention, embeddings, normalization, output heads, caches, or undeclared bypasses. The whole-model gate covers authorization to invoke those components as an atomic unit but does not partition them internally. Cargo authorization and keyed permutation conjugation are supplementary runtime and reparameterization studies, not extensions of the FFN theorem. Generative tenant-state training, heterogeneous private-corpus validation, and multi-regime internal isolation of a complete shared model remain open. The paper, Lean proofs, source-visible research preflight, experiment code, and artifacts are available at https://github.com/sekosai/schemen-gate/tree/main/research/cdp. Live identity, key-management, model-serving, network, and audit integrations require separately pinned deployment evidence.

References

M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. Deep learning with differential privacy. In *CCS*, 2016. https://arxiv.org/abs/1607.00133

L. Chen, Z. Ye, Y. Wu, D. Zhuo, L. Ceze, and A. Krishnamurthy. Punica: Multi-tenant LoRA serving. *MLSys*, 2024. https://arxiv.org/abs/2310.18547

J. Cohen, E. Rosenfeld, and Z. Kolter. Certified adversarial robustness via randomized smoothing. In *ICML*, 2019. https://arxiv.org/abs/1902.02918

T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *NeurIPS*, 2022. https://arxiv.org/abs/2208.07339

T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. In *NeurIPS*, 2024. https://arxiv.org/abs/2305.14314

N. Elhage, T. Hume, C. Olsson, et al. Toy models of superposition. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/toy_model/index.html

W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models. *JMLR*, 2022. https://arxiv.org/abs/2101.03961

E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023. https://arxiv.org/abs/2210.17323

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022. https://arxiv.org/abs/2106.09685

W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023. Apache-2.0 license. https://arxiv.org/abs/2309.06180

D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *ICLR*, 2021. https://arxiv.org/abs/2006.16668

X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *ACL-IJCNLP*, 2021. https://aclanthology.org/2021.acl-long.353/

S. Ma, H. Wang, L. Ma, et al. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv:2402.17764*, 2024. https://arxiv.org/abs/2402.17764

A. Mallya and S. Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *CVPR*, 2018. https://arxiv.org/abs/1711.05769

A. Mallya, D. Davis, and S. Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *ECCV*, 2018. https://arxiv.org/abs/1801.06519

R. McCormick. Multi-tenant LoRA: Attention is all you bleed. *In preparation*, 2026.

Predibase. LoRAX: Multi-LoRA inference server. Apache-2.0 license. https://github.com/predibase/lorax, 2024.

V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv:1910.01108*, 2019. Apache-2.0 license. https://arxiv.org/abs/1910.01108

S. A. Seshia, A. Desai, T. Dreossi, D. J. Fremont, S. Ghosh, E. Kim, S. Shivakumar, M. Vazquez-Chanlatte, and X. Yue. Formal specification for deep neural networks. In *ATVA*, 2018. https://arxiv.org/abs/1710.01688

Y. Sheng, S. Cao, D. Li, et al. S-LoRA: Serving thousands of concurrent LoRA adapters. In *MLSys*, 2024. https://arxiv.org/abs/2311.03285

SPIFFE Project. SPIFFE Workload API specification. Cloud Native Computing Foundation, 2026. https://spiffe.io/docs/latest/spiffe-specs/spiffe_workload_api/

F. Tramèr and D. Boneh. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. In *ICLR*, 2019. https://arxiv.org/abs/1806.03287

Z. Wang and J. Li. Differentially private federated learning with LoRA. *arXiv:2306.05908*, 2023. https://arxiv.org/abs/2306.05908

M. Wortsman, V. Ramanujan, R. Liu, A. Kembhavi, M. Rastegari, J. Yosinski, and A. Farhadi. Supermasks in superposition. In *NeurIPS*, 2020. https://arxiv.org/abs/2006.14769

X. Zhang, J. Zhao, and Y. LeCun. Character-level convolutional networks for text classification. In *NeurIPS*, 2015. Non-commercial research license. https://arxiv.org/abs/1509.01626